Università degli Studi di Torino

DIPARTIMENTO DI INFORMATICA
Corso di Laurea Triennale in Informatica

TESI DI LAUREA

An experimentation with SCHED_EXT Linux scheduling class

Candidato:
Matteo Di Noia
Matricola 979620

Relatore:
Enrico Bini
University of Turin

Correlatore:
Michele Castrovilli
University of Turin

Anno Accademico 2023/24

Contents

Acknowledgments **v**

Abstract **vi**

1 Basics of Linux kernel **1**
1.1 Introduction . . . 1
1.2 A general overview . . . 2
1.2.1 System calls . . . 3
1.2.2 A different kind of software . . . 4
1.2.3 User and kernel stacks . . . 5
1.2.4 A monolithic design . . . 6
1.3 Process management . . . 9
1.3.1 Processes and threads . . . 9
1.3.2 Scheduling . . . 13
1.3.3 CPU Idle Management . . . 17
1.4 Timekeeping mechanisms . . . 20
1.4.1 Time-Stamp Counter (TSC) . . . 21
1.4.2 Tasks lifecycle . . . 22

2 The Linux scheduler **25**
2.1 Elements of scheduling decisions . . . 25
2.1.1 Domains . . . 25
2.1.2 Classes and policies . . . 26
2.1.3 Priorities . . . 27
2.2 Multiprocessing . . . 28
2.2.1 Load balancing . . . 29
2.3 Scheduling domains . . . 31
2.3.1 Load-balancing threshold . . . 31
2.3.2 Flags . . . 32
2.4 Objectives of the scheduler . . . 33
2.4.1 Different workloads . . . 33
2.5 Prior to CFS . . . 35
2.5.1 The $O(1)$ Scheduler . . . 35
2.6 Completely Fair Scheduler (CFS) . . . 37
2.6.1 Weight function . . . 38

2.6.2 Runqueue 40
2.6.3 Runqueue internals 41
2.7 Scheduling Statistics 43

3 SCHED_EXT: extending the Linux scheduler 45
3.1 Current state 45
3.1.1 Installing the patched kernel 46
3.1.2 Motivations 47
3.1.3 Public interest and uses 49
3.1.4 Main issues 49
3.2 SCX Repository 50
3.3 Co-existence with other scheduling classes 51

4 Loading kernel code at runtime with eBPF 52
4.1 Safety and speed concerns 52
4.2 Layer of a SCHED_EXT scheduler 53
4.2.1 eBPF verifier 54
4.3 Definition of a SCHED_EXT scheduler 55
4.3.1 Operational structure 55
4.3.2 Dispatch queue 56
4.3.3 Callbacks 56
4.3.4 Functions 57
4.4 Scheduling cycles 58

5 Experimenting with SCHED_EXT 61
5.1 Building SCX framework 61
5.2 Testing the example 61
5.2.1 Explanation of examples present 62
5.3 Writing custom scheduler 63
5.3.1 The code to load a scheduler 63
5.3.2 The code to define a scheduler 66
5.4 Writing custom workload for the scheduler 69

6 KernelShark 73
6.1 Tracing 73
6.1.1 Event tracing with trace-cmd 74
6.2 KernelShark 75
6.2.1 Usage of KernelShark 75

List of Figures

1.1 Kernel space (in red) and user space (in green and blue) 3
1.2 The kernel stack inside its small address space of two pages, it grows downward towards low memory. 6
1.3 The most popular kernel designs and their differences 7
1.4 Kernel subsystems map[22]. The labels in the picture are structs, functions or source code files. 10
1.5 Pid hash table, pids 199 and 29384 are both hashed to 199 . . . 13
1.6 Hash table for the TGID pid type 14
1.7 State machine of task states 24

2.1 Scheduler domains' hierarchy 26
2.2 A complete domains' hierarchy 27
2.3 Local and foreign memory in a NUMA architecture 30
2.4 Nice to timeslice conversion 37
2.5 Plot of the weight as function of the nice value. 39
2.6 CPU assignment plot 40

4.1 Flowchart of scheduling a task from wake [26] 58
4.2 Flowchart of finding a task to run on a CPU [26] 59

5.1 Resulting trace visualization using Kernelshark 72
5.2 Region in which the two task's time slice changed 72

6.1 Output of the command *trace-cmd record* 74
6.2 Output of the command *trace-cmd report* 74
6.3 Record's tab of KernelShark 76
6.4 Step one of the *sched_switch* 76
6.5 Step two of the *sched_switch* 77
6.6 Step three of the *sched_switch* 77
6.7 An example of preemption 78
6.8 Step one migrate task 79
6.9 Step two, migrate task 79

List of Tables

2.1 Scheduling classes and policies in Linux 34

Acknowledgments

Part of Chapter 1 is borrowed from the thesis by Marco Perronet [21], Stefano Chiavazza [10], and Stefano De Venuto [14]. Part of Chapter 2 is borrowed from the thesis by Marco Perronet [21], Stefano Chiavazza [10], Francesco Ciraolo [11] and Antonio Casoli [8]. Part of Chapter 4 is borrowed from the thesis by the articles SCX repository documentation [13] and by the LWN article [12]. Part of Chapter 6 is borrowed from the thesis by Lorenzo Brescia [7], Giuseppe Eletto [16] and Marco Perronet [21].

I'd to thank my family, friends, supervisor and co-supervisor.

Declaration of originality:

> *"I declare to be responsible for the content I'm presenting in order to obtain the final degree, not to have plagiarized in all or part of, the work produced by others and having cited original sources in consistent way with current plagiarism regulations and copyright. I am also aware that in case of false declaration, I could incur in law penalties and my admission to final exam could be denied"*

Abstract

Chapter 1 is an introduction to the Linux operating system and its processes' management. Chapter 2 explains what a scheduler is and what characteristics it has; it also details the scheduler that are/were used in Linux. Chapter 3 is an introduction to SCHED_EXT focused on the current state of the patches and its pro and cons. Chapter 4 illustrates eBPF, the underlying language that make sched_ext possible, and how to use it to write external schedulers. Chapter 5 goes over the practical test we performed, explaining in particular an example of simple external scheduler. Chapter 6 introduces to the usage of trace-cmd command to create trace of processes and the KernelShark GUI tool to display such trace (used in previous chapter).

Chapter 1

Basics of Linux kernel

1.1 Introduction

The operating system (OS) comprises the software intended to manage the hardware resources and the *application software*, which performs specific, high-level tasks. The application software, which is the larger part of the OS, is made of utility programs and any other software with which the user interacts directly. These programs are not part of the core OS. Rather, they are necessary to do anything useful. The operating system acts as an intermediary between the user and the machine by abstracting away the hardware, which makes interaction easier: this is why almost every computer runs an operating system.

It can be argued that the OS is not strictly necessary because it's possible to execute a program without loading an OS: this is referred as *bare metal programming* and it is common in small size embedded systems. Because there is no operating system (which means no file system, memory management or any useful application such as compilers), programs cannot be written on the system itself. Instead, the program is written on another machine with an operating system, then compiled with a cross-compiler, which compiles for a target architecture different from the one it is running on. Finally, the compiled binaries are loaded at boot time on the target embedded system. This is the opposite of what we are used to doing on our laptops/desktops: to be able to reprogram the machine as it is running, by writing and compiling our program with *application software* designed to edit text and compile code. Thus, an operating system greatly simplifies interaction with the machine by offering a platform for the user and, at a higher level, by making general-purpose computing possible.

Windows, MacOS, iOS, Android... Most of us are familiar with these operating systems. Besides the platform on which they run, they are all general-purpose and their goal is the same. What really changes among them is the architecture and philosophy in their design. At a macroscopic level they differ in kernel design approach (monolithic kernel vs. hybrid kernel). This is explained later in Section 1.2.4. At a microscopic level, there is literally not much to see

because the code of most OSes is closed, so it's impossible to see the implementation differences with Linux. This leads us to one of the peculiarities of Linux: it is completely open source and community developed. Besides ethical matters, which are not discussed here, this means that it's possible to study the code and get a full understanding of operating systems. In fact, before Linux, there was no way to see how operating systems work in practice. The only option was to study them from textbooks in order to implement your own kernel, which is exactly how Linux development was initiated.

As stated earlier, a key component of an OS is "the software intended to manage the hardware resources": this is what we refer as the *kernel*. Dennis Ritchie, among the inventors of Unix and C, also called it the "Operating system proper"[25], which most likely means "The component that is the actual operating system". On the one hand this definition makes sense, because the low level tasks performed by the kernel are essential (and also because it is the most difficult component to develop). On the other, without application software the kernel is useless. In such scenario, the kernel is loaded at boot time, then it initializes and starts running, but there is nothing but a black screen because there is no other program to start. It is clear that the kernel is not an operating system by itself, but what Dennis Ritchie meant is that when we think about the core architecture of an OS, we think about the kernel. An engine is indeed useless without the rest of the car, but does that make the other components as important as the engine, where all the complexity resides? Despite the application software being the largest part of the OS; it is within the kernel that the hardest engineering challenges are found, which makes it the most interesting — and difficult — part to understand, analyze, and develop.

1.2 A general overview

The kernel's job is to manage hardware resources, which means handling all interactions with the CPUs, the memory hierarchy and the I/O devices. More specifically, the kernel needs to respond to I/O requests, manage memory allocation and decide how the CPU time is shared among the demanding processes. To achieve this, it has access to all resources in the system, which is needed to make the most out of the hardware. Its performance is what makes the difference between a fast or a slow operating system. This critical role requires a protection mechanism to ensure the stability and the security of the whole system. This is achieved by separating kernel code and user application code. In practice, depending on the configuration settings at compile time, the following steps are taken.

1. The kernel binary image is loaded in RAM in a memory area which can start from a low or high address.
2. A pre-defined slice of RAM next to that memory area is reserved to the kernel.
3. The remaining part of the memory is accessible to the user.

These two portions of the address space are called kernel space and user space. The former is a reserved area dedicated to critical system tasks and it is protected from user access. The latter is the area where system utilities and user programs run. This memory partitioning makes sure that kernel and user data do not interfere with each other. Also, it is a security measure to prevent that a malfunctioning or malicious user program may affect the entire system.

1.2.1 System calls

By extension of this design, the interaction with the user space is regulated with a privilege system. Each process can run either in user mode or kernel mode. Processes running in user mode can access privileged kernel functionalities through special gates in a pre-defined and controlled manner. These gates are implemented as functions called *system calls*, which serve as APIs between user and kernel space. When a user process performs a system call:

1. it temporarily executes in kernel mode,
2. it performs tasks that require a high privilege, and finally
3. it switches back to lower privileged user mode.

This mechanism exploits the availability of hardware functionalities. For example, in the x86 architecture 2 bits in the code selector (`cs`) register indicate the current privilege level (CPL) of the program that is running on the CPU. This value is 0 or 3, respectively, for kernel and user mode and each system call changes this value temporarily.

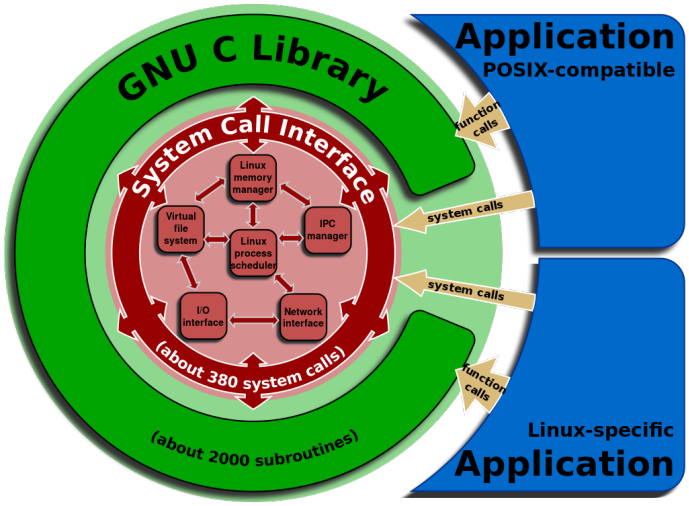


Figure 1.1: Kernel space (in red) and user space (in green and blue)

Very often, useful operations in the system require privileged services provided by the kernel. For example, even an extremely simple shell command such as `echo` performs dozens of system calls, which are reported below as listed by the command `strace -wc echo`.

```
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 29.57    0.000514         514         1           execve
 17.03    0.000296          33         9           mmap
 11.62    0.000202          67         3           brk
  8.52    0.000148          37         4           mprotect
  7.19    0.000125          25         5           close
  6.10    0.000106          35         3           open
  5.93    0.000103          26         4           fstat
  5.58    0.000097          32         3         3 access
  2.82    0.000049          49         1           munmap
  2.24    0.000039          39         1           write
  1.84    0.000032          32         1           read
  1.55    0.000027          27         1           arch_prctl
------ ----------- ----------- --------- --------- ----------------
100.00    0.001738                    36         3 total
```

System calls can be called in user space applications directly, through assembly, or indirectly, by calling wrapper functions from the C standard library (`glibc`), as shown in Figure 1.1.

```c
/*
 * Two different ways of calling open/close through glibc wrapper functions
 * SYS_open and SYS_close correspond to the syscall numbers
 */
int fd = syscall(SYS_open, "example.txt", O_WRONLY);
syscall(SYS_close, fd);
fd = open("example.txt", O_WRONLY);
close(fd);
```

1.2.2 A different kind of software

The separation between kernel space and user space and the fact that we are working at such low level makes the kernel a very peculiar piece of software. One of its properties is that there is no error checking, this is because the kernel trusts itself: all kernel functions are assumed to be error-free, so the kernel does not need to insert any protection against programming errors [9]. Instead, what the kernel does is to use assertions to check hardware and software consistency; if they fail then the system goes into *kernel panic* and halts. The choice of checking assertion (and possibly going to kernel panic if something went wrong) is that since the kernel controls the system itself, error detection, recovery and correction is very hard and would take a huge part of the code. Of course,

programming or hardware errors can (and will) still occur: when this happens the offending process is killed and a memory dump called "*oops*" is created. A typical example of this happens when the kernel dereferences a NULL pointer: in user space this would cause a *segmentation fault*, while in the kernel it will generate an oops or in the worst case go directly into panic. After this kind of event, the kernel can no longer be trusted and the best thing to do would be to reboot, because the kernel is in a semi-usable state and it could potentially corrupt memory. Furthermore, a panic in this state is more likely to happen. Possibly, the user experiencing the kernel panic may also inform the kernel maintainers.

Another peculiarity of the kernel is that it uses its own implementation of the functions in the standard C library. For example `printf()` and `malloc()` are implemented as `printk()` and `kmalloc()`. There are different reasons for this choice. one is that the standard C library is too big and inefficient for the kernel. Another reason is that implementing your own functions gives more freedom because they can be customized for their purpose in the kernel. Memory allocation in user or kernel space is very different, so the `kmalloc()` implementation is very specific. For instance, kernel data structures need a contiguous physical memory segment to be allocated, while regular user space allocation does not need to obey to this restriction. Furthermore, `printk()` writes its output into the kernel log buffer (that you can read by using the `dmesg` command in user space); this is very different from `printf()` that writes on standard output.

1.2.3 User and kernel stacks

As stated earlier, the memory management is different in kernel/user space. The same happens with the execution. Every process in the system has two stacks, located respectively in user and kernel space, and it will use one of the two while executing in the corresponding privilege mode. x86 CPUs automatically switch stack pointers when privilege mode switches occur, which usually happens for syscalls. The user space stack can potentially be very big, with a very high limit, and even though it's initially small it can allocate more physical memory as it needs it: this mechanism is called "*on-demand paging*". The kernel stack, unlike the user stack, cannot expand itself and it has a fixed size of two pages. Since, 32-bit and 64-bit systems have 4KB and 8KB sized pages, then the kernel stack size is of size 8KB or 16KB, respectively. These two pages must be allocated contiguously, which can cause memory fragmentation for long system uptimes as stacks get deallocated. In other words, it becomes increasingly hard to find two physically contiguous available pages as the OS runs for a long time. For this reason, in the past, efforts were made to reduce the stack size to one page, which would eliminate fragmentation, but after many stack overflows the standard settled on two pages.

This leads us to an interesting example of the kernel trusting itself: it makes the strong assumption that the stack will never overflow: **no protection against is in place**. So what happens if the stack overflows? First, it will

corrupt the `thread_info` data structure, which is the first data that the stack encounters along its path (Figure 1.2). This will make the process nonexistent for the kernel and cause a memory leak. Next, the stack can overflow outside of the address space and silently corrupt whatever kernel data is stored; the best case scenario here would be a kernel panic to prevent any further memory corruption. Another natural question might be "why are kernel stacks so small?" and the answer is simple: first, to use a small amount of kernel memory, and secondly, because of fragmentation. The bigger is your data structure in contiguous physical memory, the more it is hard to allocate. It is expected that any process stays in kernel mode for a small amount of time, so it should use a very small portion of the stack. A consequence of small stacks is that very few recursive functions are used to avoid long call chains and minimize stack usage; the same is true for big static allocations on the stack.

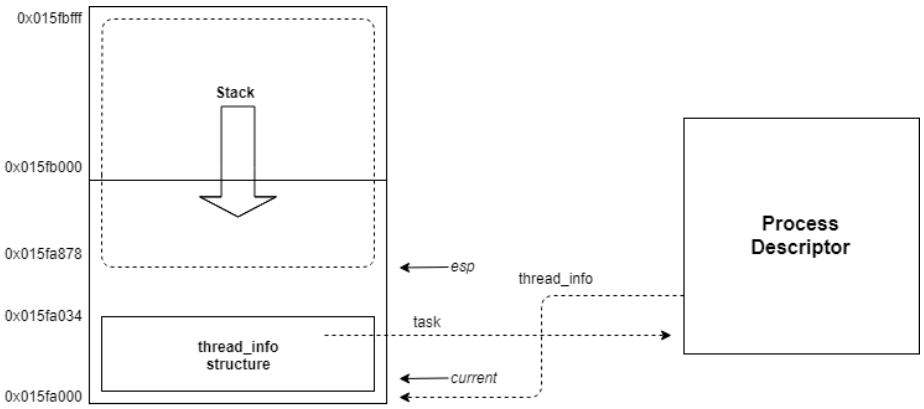


Figure 1.2: The kernel stack inside its small address space of two pages, it grows downward towards low memory.

It's important to note that there are special processes called *kernel threads* that do not follow this pattern of kernel/user stack. Kernel threads perform a specific system task, they are created by the kernel and they live exclusively in kernel space, never switching to user mode. Their address space is the whole kernel space and they can use it however they want. Besides this, they are normal and fully schedulable tasks just like the others. An example of a kernel thread is `ksoftirqd`: there is always one for each CPU and their job is to dispatch interrupt requests. As a side note, the name stands for "Kernel Software Interrupt ReQuest Daemon", many kernel threads follow a similar naming convention.

1.2.4 A monolithic design

There are different design approaches to the kernel development. We can see all of them them as a spectrum. At on one end, there is the *monolithic kernel*,

and at the other end the *microkernel* (or *μkernel*). The choice depends on how many services are located in kernel space: while in monolithic design every service is in the kernel itself, microkernels strive to reduce as much as possible the code running in kernel space. This is done by moving most services in user space, while keeping only essential primitives in the kernel (Figure 1.3).

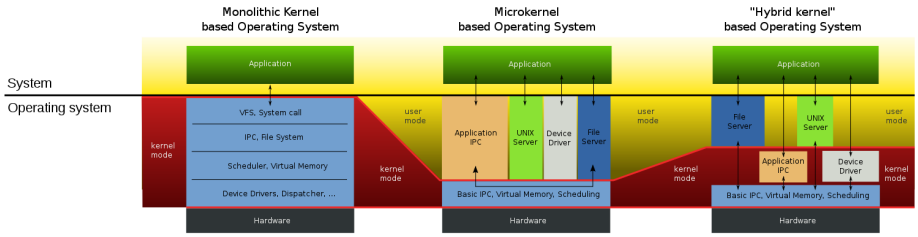


Figure 1.3: The most popular kernel designs and their differences

These services are implemented as *servers*, and communication between the servers, applications, and the kernel is based on message passing. As in classic client/server approach, applications send requests to the servers, which can in turn request services to the kernel or satisfy the request directly. Because of this design choice, the system relies heavily on *Inter-Process Communication* (IPC). Even if they are part of the core architecture, the servers are user processes and run in user space just like the other user processes, though they get higher privileges.

By reducing the code running in kernel space, there is less risk for bugs. Because the trusted codebase is very small, there is no need to make big assumptions like in monolithic kernels. As stated before, a bug in the kernel can bring down the entire system, but in microkernels bugs are contained. For example, if the networking service crashes, then we can just restart it since it's just a user process; in a monolithic environment, this problem would have crashed the entire system: this is one of the biggest flaws of the monolithic design. A small trusted codebase also means more portability because all the architecture dependent code is concentrated in the small kernel. The actual operating system is built on top of it, so it would be possible to implement it in a higher level language, while only the primitives in the kernel must be ported. Conversely, in a monolithic kernel, many functions must be rewritten for each architecture. In Linux, the folder for architecture dependent code (`arch`) is the second largest folder and it represents 8% of the code.

Another direct consequence of the shift of the code in user space is that microkernels are more easily maintainable. Development is easier because most of the code runs in user space, so the usual restrictions for kernel code are not present: for example, it would be possible to make use of `glibc`. Furthermore, testing can be done without rebooting the system: just stop the service, recompile the code and then start it again. On a monolithic system, not only it's needed to recompile the whole kernel, we must also reboot in order to load the image again. And if this new image doesn't work, then we must reboot again

with the working image. In practice, this is always done in a virtual machine in order to test more efficiently, but it's still a tedious process.

Given all these advantages, why aren't microkernels always used? It is mostly because of one severe issue: the performance penalty. It's easy to see this if we think that monolithic kernels communicate directly with hardware, while in microkernels most of the operating system does not; essentially, microkernels add an additional layer of abstraction through heavy use of IPC. More precisely, the task of moving in and out of the kernel to move data between applications and servers creates significant overhead. This process results in two major problems:

1. A large number of system calls, caused by services frequently needing to use the primitives.

2. Many context switches, because each service must be scheduled as a process. In order to pass a message between two services, a full context switch is needed to send and receive.

This last problem is not an issue in a monolithic setting, because kernel functions are executed when any currently running process enters kernel space. Of course, calling a plain function is much less costly than doing a system call or context switch. Furthermore, IPC in monolithic kernels is implemented through shared memory, which is more efficient than IPC with message passing. In Linux, because every functionality is in the kernel, it is a single, big program running in his dedicated address space: this means that every subsystem (scheduling, IPC, networking, memory management...) shares the same memory. Paradoxically, all the auxiliary code needed for interfacing and communication can make microkernel-based operating systems larger than monolithic kernels, even though all this code is not in kernel space.

Linux is a monolithic kernel, and because of this design choice, even the device drivers are located in kernel space: in fact, more than 65% of the kernel code is just drivers (in the `driver` directory, the largest folder). This means that while the system is running a huge part of the code is not being used. For this reason, many miniaturized versions of Linux have been distributed: a fully functional — and still monolithic — kernel can fit on a single floppy disk. If we wanted to create just a reduced version, it would not be too hard to remove drivers that are not needed and then recompile the kernel.

A problem of the monolithic design is the natural lack of modularity; microkernels do not have this problem because it's very easy to start/stop drivers running in user space. Monolithic kernels try to achieve the modularity of microkernels by using *kernel modules*: they are simply code that can be inserted/removed from the kernel at runtime. A module can be linked to the running kernel when its functionality is required and unlinked when it is no longer needed. This is very useful for small embedded systems, to keep running code to a minimum. Modules are often used to add/remove drivers and this approach is much faster than having drivers in user space. Since the code runs in kernel space, there is no need to do message passing or communicate

with user space at all. It is the same as in a microkernel, but without the performance penalty. This comes at the price of programming in kernel space, which is harder. In the end, it is a choice between ease of development/fault tolerance or performance. Furthermore, modules, unlike the external layers of microkernel operating systems, do not run as a specific process. Instead, they are executed in kernel mode on behalf of the current process. Like any other kernel function: this means less switching between processes, and then better performance. Because of the big issue of monolithic kernels (of having a large code base), if a driver module behaves incorrectly, the system can crash upon module insertion.

Modules are powerful, but cannot always accomplish what a microkernel can. As an example: on Linux, it's not possible to replace the scheduler at runtime. In order to do that, it's needed to have the two different schedulers directly in the core code and switch between them at runtime. This is how it's actually done in Linux: different schedulers are already implemented and available. Usually, modules are not used to implement core functions. Rather, they are viewed as extensions of the kernel. This means that it is very difficult to modify policies decided by the kernel through modules, and users must adapt to these policies or modify the code and recompile the whole kernel. Conversely, in microkernels it is simpler to change the core implementation, since it runs as a service.

Finally, the hybrid design is halfway between monolithic and microkernel, as it tries to take the best side of both approaches: having good performance, but also, to some extent, flexibility and maintainability. In practice, its philosophy is very similar to a monolithic kernel. Notable OSes that use a hybrid kernel are Windows and MacOS.

1.3 Process management

The kernel is divided into subsystems that interact with each other. Figure 1.4 is a zoom into the kernel mechanisms inside the red part of Figure 1.1.

1.3.1 Processes and threads

A process is an instance of a running program. Each process has resources associated with it, such as an address space, open files, global variables and code. Each process must have its own address space that it can only access. When a process tries to access a memory location that does not belong to it, a *segmentation fault* interrupt is generated and a signal `SIGSEGV` is delivered to the process. A thread is defined as a single flow of execution, it has associated a stack of execution and the set of CPU registers that it uses, most notably the stack pointer and program counter. Each process can have multiple threads, in which case it is a *multi-threaded process*. Threads belonging to the same process will share resources among each other. The execution aspect of a process is

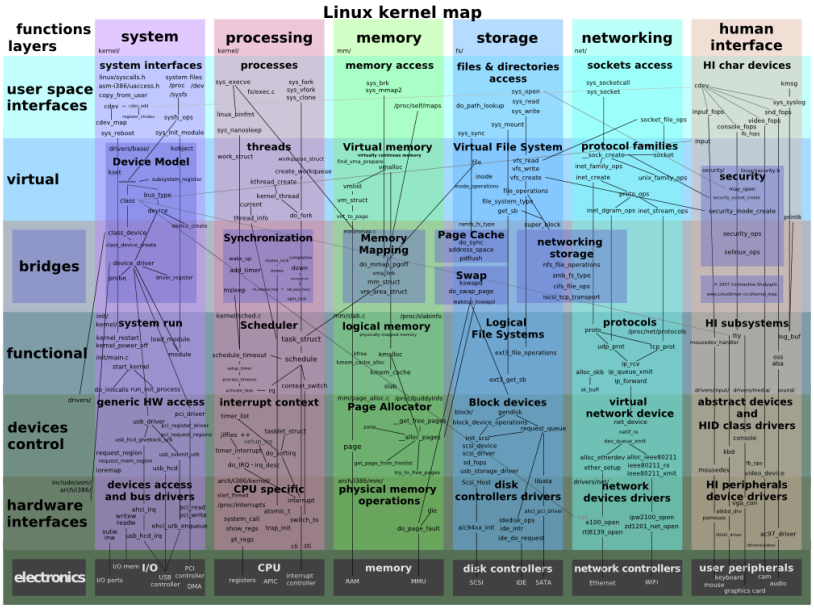


Figure 1.4: Kernel subsystems map[22]. The labels in the picture are structs, functions or source code files.

always represented by threads, which means that a process cannot exist without at least one thread associated.

The kernel does not distinguish between processes and threads, so they are treated as the same entity. Because of this, a problem in terminology arises. Next, it is shown how processes ans threads are distinguished.

Each process has its own PID (Process IDentifier) and groups of processes are identified by the TGID (Thread Group ID). If a process only has a single thread then its PID is equal to its TGID. If a process is multi-threaded then each *thread* has a different PID, but they will all have the same TGID. Furthermore, there will be a thread in this group called *thread group leader* that will have its PID equal to the TGID, so the TGID field in each thread is just the PID of their leader. Just to add some more confusion, when you call `getpid()` you are actually getting the TGID (the group leader PID, identifying the whole process), and when you call `gettid()` you are getting the PID (which identifies a single thread, not a group). Hence, the PID resembles more a thread identifier.

This confusing way of using IDs was implemented to comply with POSIX standards, which require that each thread of a multi-threaded process must have the same id: this is why `getpid()` returns the TGID.

You can already see how "process" and "thread" are being used interchangeably. The real difference between threads and processes is that threads share the same address space, while processes do not. This means that threads associated to a same process can communicate more simply that different processes. This

enables concurrent programming, it enables communication among threads via shared memory, and it requires then synchronization methods. The usage of threads in a program rather than spawning new processes, results in much better performance, which is why threads are sometimes called *lightweight processes* or LWP.

As illustrated next, the Linux-specific system call `clone()` spawns a new child process.

```
1  /* stack size for cloned child */
2  const int STACK_SIZE = 65536;
3  /* start and end of stack buffer area */
4  char *stack, *stackTop;
5  /* ... define child startup function "do_something" ... */
6  stack = malloc(STACK_SIZE);
7  stackTop = stack + STACK_SIZE; //stack grows downward
8
9  /* spawns a new thread */
10 clone(&do_something, stack + STACK_SIZE, CLONE_VM | CLONE_FS | CLONE_FILES |
   ↪  CLONE_SIGHAND, 0);
11 /* spawns a new process, this is the same as using fork() */
12 clone(&do_something, stack + STACK_SIZE, SIGCHLD, 0);
```

The system call `clone` is very similar to `fork()`, but it offers a variety of flags to configure what type and how many resources are shared with the new process. For example, `CLONE_VM` (where `VM` stands for virtual memory) makes the child process run in the same address space as the parent, while the other flags clone filesystem information (such as working directory), open files and signal handlers. The flag `SIGCHLD` at line 12 requires that the parent process receives a `SIGCHLD` signal upon the termination of the created child process. Ultimately, the reason why threads and processes are treated as the same entity in Linux, is that processes are just threads that share nothing. In fact, the word *task* is always used inside the kernel instead of process/thread.

Each task is represented in the kernel with the struct `task_struct`, this is a fairly big structure that can be almost 2KB in size, depending on configuration at compile time. The `task_struct` is what is often referred as the *process descriptor* or PCB (*process control block*): every information about a task is stored in here.

```
1  /* Code from ./include/linux/sched.h */
2  struct task_struct {
3          /* -1 unrunnable, 0 runnable, >0 stopped: */
4          volatile long state;
5          void *stack;
6          /* Current CPU: */
7          unsigned int cpu;
8          /* A boolean, "on_runqueue" */
9          int on_rq;
10         int prio;
11         int static_prio;
12         int normal_prio;
13         int exit_state;
```

```
        int exit_code;
        int exit_signal;
        /* The signal sent when the parent dies: */
        int pdeath_signal;
        pid_t pid;
        pid_t tgid;
        /* Real parent process: */
        // The original parent that forked this task
        struct task_struct __rcu *real_parent;
        /* Recipient of SIGCHLD, wait4() reports: */
        // The current parent, maybe the original one exited
        struct task_struct __rcu *parent;
        // Executable name, usually the command that spawned this task
        char comm[TASK_COMM_LEN];
        /* Filesystem information: */
        struct fs_struct *fs;
        /* Open file information: */
        struct files_struct *files;
        /*
         * Children/sibling form the list of natural children:
         */
        struct list_head children;
        struct list_head sibling;
        struct task_struct *group_leader;
        /* PID/PID hash table linkage. */
        struct pid *thread_pid;
        struct hlist_node pid_links[PIDTYPE_MAX];
        struct list_head thread_group;
        struct list_head thread_node;
};
```

The code above shows some of the most basic fields the struct, most of them are self-explanatory.

The `volatile` keyword requires the compiler not to optimize by caching the storage of this variable. This indicates that the value may change even if the variable does not appear to have been modified. Hence, every time a `volatile` variable is accessed, it needs to be read from the main memory. The opposite of `volatile` is the compiler hint `register`. The fact that the task state is volatile makes sense because it could be unpredictably modified by interrupts: it could be possible that an old value of the variable is read from the cache instead of the actual value.

Let us now focus on the `pid` fields to show how Linux uses pids to find any information and resources of tasks. Given a pid, searching linearly through the pids to find the task we are looking for would be very inefficient. Instead, a hash table known as *pid hash table* is used for this purpose. The identifiers in this table are simply the result of hashing a given pid, you can see in figure 1.5 that conflicting entries are simply stored in a list associated with the same id. Because it's a hash table, the kernel can quickly look up the pid and find in $O(1)$ time the corresponding process descriptor. This procedure is, for example, applied when the command `kill [PID]` is launched.

There are actually four tables, one for each PID type, which can be:

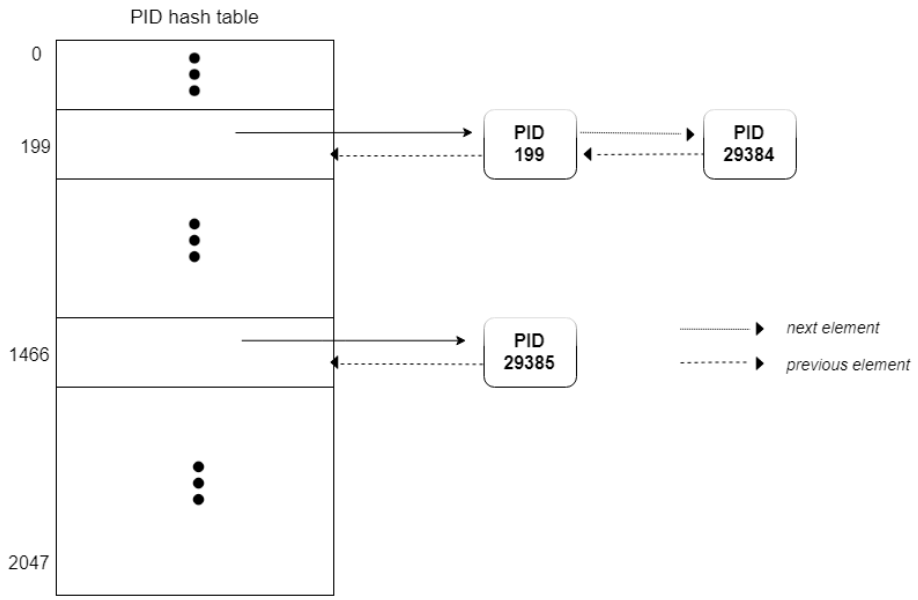


Figure 1.5: Pid hash table, pids 199 and 29384 are both hashed to 199

- `PID`, a process ID
- `TGID`, a thread group leader PID
- `PGID`, a process group leader PID
- `SID`, a session leader process PID

Each of these tables is an array of `hlist_head`, the head of the chain list, which points to a list of `hlist_node` (see Figure 1.6). These structures are used for non-circular lists. These lists are populated by `struct pid`, and a pointer to this struct is stored inside each process descriptor in the `thread_pid` field. Figure 1.6 shows an example for the TGID class illustrated earlier.

1.3.2 Scheduling

A system with a single CPU can execute only one process at a time. For this reason, a scheduler for processes is needed. Process scheduling consists in choosing which processes should run in what order, essentially deciding how CPU time is shared among processes. To achieve this, there are many scheduling algorithms such as FCFS (*first come first served*), RR (*round-robin*), EDF (*earliest deadline first*), SJF (*shortest job first*), FP (*fixed priorities*), and more. Most of the scheduling policies are *preemptive*, which means that at any time the scheduler can arbitrarily decide to interrupt the currently running task and assign the CPU to another process. The use of preemption implies that processes have

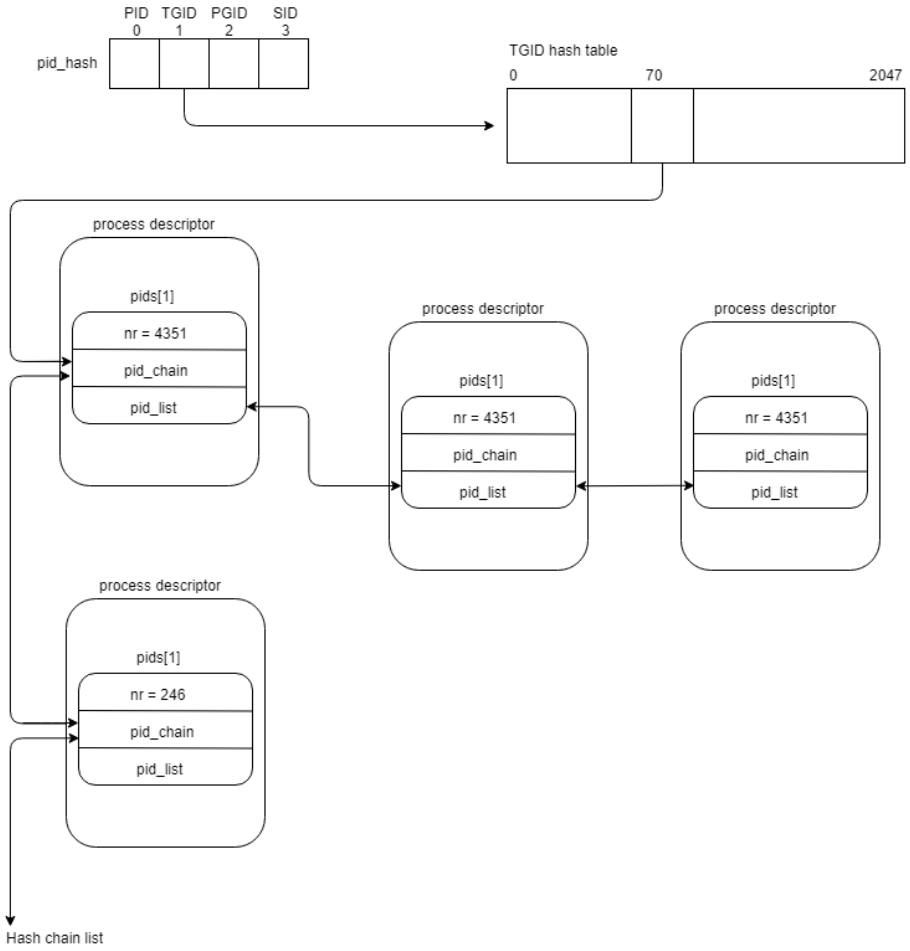


Figure 1.6: Hash table for the TGID pid type

assigned *timeslices*: they are periods of time in which the process is allowed to run and after which it will be preempted.

FCFS, which is the most basic scheduling algorithm, doesn't have preemption nor timeslices: every process runs as much as it wants before voluntarily giving up the CPU to the next task in the queue. Round-robin is similar to FCFS because it has a FIFO runqueue; the difference is that it uses a constant timeslice, called *quantum*, assigned to each process: when the quantum expires the process gets preempted and is moved to the tail of the queue. Then, the next one is scheduled.

In a (*uniprocessor*) system it is not possible to achieve true parallelism among processes. The only way to do it is to have multiple processors that communicate through a shared bus and the central memory: this is known as SMP (*symmetric*

multiprocessing). A single processor can also have multiple cores, but each one is treated as a separate processor, so the SMP architecture applies to cores as well. Even on SMP systems, which represent most systems today, there often are more processes than cores. Hence, scheduling is necessary for each processor/core. There are also new problems that arise in SMP, such as *load balancing*: the problem of balancing processes between CPUs so that no CPU goes idle or has an unfair amount of workload. This kind of related problems must also be taken into account by the scheduler.

Every job carried out by the scheduler will eventually lead to a process switch on a given CPU. The kernel has a mechanism to suspend the execution of a process, save its status, and resume another process. This procedure is called *context switch*. Each process has an *execution context*, which includes everything needed for a task to execute, from its stack to the code. While every process can have its own process descriptor, the registers on the CPU must be shared between every process in the system. Every value in any register that a process is using is a subset of the execution context and is called the *hardware context*. At every context switch the hardware context must be saved and restored, for the old and the new process, respectively. The content of the registers are saved in part in the process descriptor of the preempted process, and in part on its kernel stack.

The routine that performs a context switch is called, not surprisingly, `context_switch()`, and it is called only in one well-defined point in the kernel: inside the `schedule()` function. Such an invocation triggers the scheduler and chooses the next task to schedule. `context_switch()` switches the address spaces of the two processes and then calls `__switch_to()`. This last function operates on registers and kernel stacks, so it is one of the most architecture dependent in the whole kernel. This is why, like many other similar routines, there is one version for each architecture supported by Linux in the `arch` folder.

There are 6 *segmentation registers* that hold *segment selector*, basically the starting address of memory segments in the process address space.

- `cs` *Code segment*, this points to the segment containing instructions of the loaded program, also known as the `.text` section. As mentioned in Section 1.2.1, this register also holds 2 bits that describe the current privilege level of the CPU.
- `ss` *Stack segment*, points to the segment containing the stack of execution.
- `ds` *Data segment*, points to the segment containing global variables and constants, also known as the `.data` section.

The other 3 segments, `es`, `fs` and `gs` are general-purpose and do not hold a specific address. There are also general-purpose data registers that hold data used in operations (`ax`, `bx`, `cx`, `dx`) and pointer registers, that hold offsets:

- `ip` *Instruction pointer*, offset to the next instruction. If added to `cs` will be the address of the next instruction to fetch (`cs:ip`).

- `sp` *Stack pointer*, offset to the top of stack. If added to `ss` will be the address of the top of stack (`ss:sp`).
- `bp` *Base pointer*, offset to subroutine parameters on the stack (`ss:bp`).

Let us now review which part of the process descriptor is involved in context switching.

```
struct task_struct {
    /* ... */
    /* CPU-specific state of this task: */
    struct thread_struct    thread;
};
```

```
struct thread_struct {
#ifdef CONFIG_X86_32
    unsigned long       sp0;
#endif
    unsigned long       sp;
#ifdef CONFIG_X86_32
    unsigned long       sysenter_cs;
#else
    unsigned short      es;
    unsigned short      ds;
    unsigned short      fsindex;
    unsigned short      gsindex;
#endif
    /* ... */
    /* Floating point and extended processor state */
    struct fpu      fpu;
};
```

This struct is very architecture dependent as it depends on the hardware registers. Its purpose is to save the hardware context before the context switch. Even if it is specific to x86, it can still change depending on whether the architecture is 32 or 64 bits. Notice that only a small part of the hardware context is saved in the process descriptor: the kernel stack pointer, general-purpose segmentation registers, data segment and the floating point registers. In older versions of the kernel most of the registers were stored here.

Let us now illustrate what happens when the kernel switches from process A to process B. There are actually two different mechanisms in this procedure: the entry/exit mechanism (user/kernel stack switch) and the context switch.

1. Process A enters kernel mode, so it will switch from its user stack to its kernel stack, in other words: it saves its **user** hardware context in the kernel stack. It does so by pushing its **user mode** stack (`ss:sp`), instruction pointer (`cs:ip`) and data registers onto the kernel stack, then all CPU registers are switched to use the kernel stack.

2. When in kernel context, process A invokes `schedule()` which will eventually do `context_switch()`.

3. Process A saves its hardware context:
   (a) It pushes most of its register values onto the kernel stack by a series of `mov` assembly operations.
   (b) It saves the value of the stack pointer (which is pointing to the **kernel** stack) into its `task_struct->thread.sp`.
   (c) Other registers such as the floating point registers are saved in the `thread` field of `task_struct`.
4. Process A loads a previously saved stack pointer from process B's `task_struct->thread.sp`, also loads the other saved registers
5. Address spaces are switched.
6. Using the loaded stack pointer, process B moves its previously saved registers from its kernel stack into the registers. This is done by a series of `pop [register]` assembly operations. Process B's state is now completely restored.
7. process B exits kernel mode and restores its **user** context. This is accomplished by loading previously saved registers from the kernel stack: its **user mode** stack (`ss:sp`), instruction pointer (`cs:ip`) and data registers. Process B is now in user context.

1.3.3 CPU Idle Management

Logical CPUs, simply referred to as "CPUs" in what follows, are regarded as **idle** by the Linux kernel when there are no tasks to run on them except for the special "idle" task.

Tasks can be in various states. In particular, they are **runnable** if there are no specific conditions preventing their code from being run by a CPU as long as there is a CPU available for that (for example, they are not waiting for any events to occur or similar). When a task becomes runnable, the CPU scheduler assigns it to one of the available CPUs to run and if there are no more runnable tasks assigned to it, the CPU will load the given task's context and run its code (from the instruction following the last one executed so far, possibly by another CPU).

The special "idle" task becomes runnable if there are no other runnable tasks assigned to the given CPU and the CPU is then regarded as idle. In Linux idle CPUs run the code of the "idle" task called the **idle loop**. That code may cause the processor to be put into one of its idle states, if they are supported, in order to save energy, but if the processor does not support any idle states, or there is not enough time to spend in an idle state before the next wakeup event, or there are strict latency constraints preventing any of the available idle states from being used, the CPU will simply execute more or less useless instructions in a loop until it is assigned a new task to run.

The idle loop code takes two major steps at every iteration. First, it calls into a code module referred to as the **governor** that belongs to the CPU idle time management subsystem called *CPUIdle* to select an idle state for the CPU to ask the hardware to enter. Second, it invokes another code module from the *CPUIdle* subsystem, called the **driver**, to actually ask the processor hardware to enter the idle state selected by the governor.

The role of the governor is to find an idle state most suitable for the conditions at hand. For this purpose, idle states that the hardware can be asked to enter by logical CPUs are represented in an abstract way independent of the platform or the processor architecture and organized in a one-dimensional (linear) array. That array has to be prepared and supplied by the *CPUIdle* driver matching the platform the kernel is running on at the initialization time. This allows *CPUIdle* governors to be independent of the underlying hardware and to work with any platforms that the Linux kernel can run on.

There are four *CPUIdle* governors available, `menu`, `TEO`, `ladder` and `haltpoll`. Which of them is used by default depends on the configuration of the kernel and in particular on whether or not the scheduler tick can be stopped by the idle loop. Available governors can be read from the `available_governors` file, and the governor can be changed at runtime. The name of the *CPUIdle* governor currently used by the kernel can be read from the `current_governor_ro` or `current_governor` file under `/sys/devices/system/cpu/cpuidle/` in sysfs. As an example, in the current PC:

```
$ cat /sys/devices/system/cpu/cpuidle/available_governors
ladder menu teo
$ cat /sys/devices/system/cpu/cpuidle/current_governor
menu
```

Which *CPUIdle* driver is used, on the other hand, usually depends on the platform the kernel is running on, but there are platforms with more than one matching driver. The name of the *CPUIdle* driver currently used by the kernel can be read from the `current\_driver` file under `/sys/devices/system/cpu/cpuidle/` in the same filesystem as before. As an example, in the current PC:

```
$ cat /sys/devices/system/cpu/cpuidle/current_driver
acpi_idle
```

For the CPU idle time management purposes all of the physical idle states supported by the processor have to be represented as a one-dimensional array of `struct cpuidle_state` objects each allowing an individual (logical) CPU to ask the processor hardware to enter an idle state of certain properties.

```
struct cpuidle_state {
        char            name[CPUIDLE_NAME_LEN];
        char            desc[CPUIDLE_DESC_LEN];

```

```
        s64             exit_latency_ns;
        s64             target_residency_ns;
        unsigned int    flags;
        unsigned int    exit_latency; /* in US */
        int             power_usage; /* in mW */
        unsigned int    target_residency; /* in US */

        int (*enter)    (struct cpuidle_device *dev,
                        struct cpuidle_driver *drv,
                        int index);

        int (*enter_dead) (struct cpuidle_device *dev, int index);

        /*
         * CPUs execute ->enter_s2idle with the local tick or entire timekeeping
         * suspended, so it must not re-enable interrupts at any point (even
         * temporarily) or attempt to change states of clock event devices.
         *
         * This callback may point to the same function as ->enter if all of
         * the above requirements are met by it.
         */
        int (*enter_s2idle)(struct cpuidle_device *dev,
                            struct cpuidle_driver *drv,
                            int index);
};
```

The more useful fields to illustrate are:

`desc` Description of the idle state.

`latency` Exit latency of the idle state in microseconds.

`name` Name of the idle state.

`power` Power drawn by hardware in this idle state in milliwatts.

`residency` Target residency of the idle state in microseconds.

In addition to the parameters above, the objects representing idle states each contain a pointer to the function to run in order to ask the hardware to enter that state.

Also, for each `struct cpuidle_state` object, there is a corresponding `struct cpuidle_state_usage` one containing usage statistics of the given idle state.

```
struct cpuidle_state_usage {
        unsigned long long      disable;
        unsigned long long      usage;
        u64                     time_ns;
        unsigned long long      above; /* Number of times it's been too deep */
        unsigned long long      below; /* Number of times it's been too shallow */
```

```
        unsigned long long      rejected; /* Number of times idle entry was
        ↪  rejected */
#ifdef CONFIG_SUSPEND
        unsigned long long      s2idle_usage;
        unsigned long long      s2idle_time; /* in US */
#endif
};
```

And again, the more useful fields:

`disable` Whether or not this idle state is disabled.

`time_ns` Total time spent in this idle state by the given CPU in nanoseconds.

`usage` Total number of times the hardware has been asked by the given CPU to enter this idle state.

`rejected` Total number of times a request to enter this idle state on the given CPU was rejected.

All the information contained in these two structures is exposed by the kernel via *sysfs*. In fact, for each CPU in the system, there is a `/sys/devices/system/cpu/cpu<N>/cpuidle/` directory that contains a set of subdirectories, one for each idle state objects defined for the given CPU. The larger the number in its name, the deeper the (effective) idle state represented by it. Each of them contains a number of files (attributes) representing the properties seen previously.

1.4 Timekeeping mechanisms

Timekeeping is the process or activity of recording how long something takes. We need "instruments" to measure time. The Linux kernel has several abstractions to represent such devices.

Clocksource The purpose of the clock source is to provide a timeline for the system that tells you where you are in time. For example issuing the command `date` on a Linux system will eventually read the clock source to determine exactly what time it is.

Typically the clock source is a monotonic, atomic counter which will provide n bits which count from 0 to $(2^n) - 1$ and then wraps around to 0 and start over. It will ideally **never** stop ticking as long as the system is running. It may stop during system suspend.

The clock source shall have as high resolution as possible, and the frequency shall be as stable and correct as possible as compared to a real-world wall clock. It should not move unpredictably back and forth in time or miss a few cycles here and there.

Clockevent device Clock events are the conceptual opposite of clock sources. They take a desired time specification value and calculate the values to poke into hardware timer registers.

Clock events are orthogonal to clock sources. The same hardware and register range may be used for the clock event, but it is essentially a different thing.

`sched_clock()` This function is similar to clocksource, but should be "cheaper" to read (meaning that one can get its value fast), as `sched_clock()` is used for task-scheduling purposes and scheduling happens often. That means sacrificing accuracy and other characteristics for speed.

To generate a clocksource many hardware devices are available. Examples are the Programmable Interrupt Timer (PIT), the Real-Time Clock (RTC), Advanced Programmable Interrupt Controller (APIC) and High Precision Event Timer (HPET), but the detailed specification of them is beyond the current scope of this work.

A full list of clocksources available on a machine along with the one used can be retrieved by interacting with the related *sysfs*, by respectively reading the `available_clocksource` and `current_clocksource` files in the `/sys/devices/system/clocksource/clocksource0` folder. As an example, on the current machine:

```
$ cat /sys/devices/system/clocksource/clocksource0/available_clocksource
tsc hpet acpi_pm
$ cat /sys/devices/system/clocksource/clocksource0/current_clocksource
tsc
```

1.4.1 Time-Stamp Counter (TSC)

The Time-Stamp Counter (TSC) is used to count processor-clock cycles. The TSC is cleared to 0 after a processor reset and it will be incremented at a rate corresponding to the baseline frequency of the processor (which may differ from actual processor frequency in low power modes of operation). Each time the TSC is read, it returns a monotonically-larger value than the previous value read from the TSC. When the TSC has all bits equal to one, it wraps to zero. The TSC in a 1-GHz processor counts for almost 600 years before it wraps.

The counter's architecture includes the following components:

- **TSC flag** A feature bit that indicates the availability of the time-stamp counter.
- **IA32_TIME_STAMP_COUNTER MSR** The MSR used as the counter.
- **TSD flag** A control register flag used to enable or disable the time-stamp counter.

The TSC register be read using one of the special read time-stamp counter instructions, RDTSC (Read Time-Stamp Counter) or RDTSCP (Read Time-Stamp Counter and Processor ID).

The former reads the Time-Stamp Counter and is guaranteed to return a monotonically increasing unique value whenever executed, except for a 64-bit counter wraparound, the latter returns atomically not just the TSC, but also an indicator which corresponds to the processor number. This can be used to index into an array of TSC variables to determine offset information in SMP systems where TSCs are not synchronized.

Of course, the regular RDMSR and WRMSR instruction can be used, treating the Time-Stamp Counter as an ordinary MSR.

When used in virtual environments, an important mechanism can be used, the Timestamp-counter offsetting (TSC offsetting). It is a feature that allows VMM software to specify a value (the TSC offset) that is added to the TSC when it is read by guest software. A VMM can use this feature to provide guest software with the illusion that it is operating at a time later or earlier than that represented by the current TSC value.

1.4.2 Tasks lifecycle

Tasks have a life cycle: a new child process task is created every time a task uses fork-like system calls. As shown earlier in Section 1.3.1, once a process is created some resources and configurations are inherited from the parent, depending on the `clone()` flags, while `fork()` will duplicate the calling process. There are some resources that will always be inherited and there is no reason to duplicate, such as the executable object code (the `.text` memory segment in Linux). The new process will be in the runnable state and ready to be scheduled. When the process needs to wait for a particular resource, it goes into a sleep state; it will then become runnable again when the resource is available, or after a pre-defined time when the syscall `sleep()` is used. A process can also go from running to runnable: this happens if the process is preempted or if it gives up the CPU voluntarily. This last case happens, for instance, if the process needs to do I/O operations for which it doesn't need the CPU. This way no processor time is wasted and another task is scheduled.

A process terminates by executing `exit()` or when it receives a signal (including `SIGHUP`, `SIGINT`, `SIGKILL`, `SIGTERM`, and others) from other processes which have the privileges to do so. Upon exit, its *exit state* will initially be set to the *zombie* state. A zombie process is a process that terminated, but its process descriptor and entry in the pid hash table are still present in memory and accessible (for example, by `ps -aux`). Tasks' resources are not deallocated immediately because the parent process may want to access some of this information, most likely the *exit status*, or may want to synchronize with the child process termination via `wait()` or `waitpid()`. This is actually a relevant resource leak because `task_struct` is almost 2KB in size. Hence, if there are many zombie processes then a big portion of memory is simply wasted until the parent process executes a `wait()/waitpid()`. More in details, a `task_struct`

plus its kernel stack consumes around 10KB of low kernel memory, that is `THREAD_SIZE + sizeof(struct task_struct)`, assuming that kernel stacks are 8KB in size (`thread_info` and pidhash entry are too small to be relevant).

After terminating and possibly sending the signal `SIGCHLD` to the parent, a task will remain zombie until its parent performs a `wait()`, upon which the parent gets information about the terminated child. Subsequently, `release_task()` is executed and the last data structures from the descriptor get detached. `detach_pid()` is called twice to clear the entry in both the `PID` and `TGID` hash tables, then `task_struct` is finally deallocated. Zombie processes are impossible to kill externally: they can't receive signals as they no longer exists, so a wait by the parent is the only way to clean the memory occupied by the zombie data structure. Suppose that the parent of a zombie process exits without waiting: the child will be an orphan process so it will be become a child of `init`. Luckily, the ancestor process (`init`) has a routine that waits periodically to reap possible zombie processes; so the child process will simply be waited by `init` and get cleared. This mechanism ensures that memory won't be cluttered by zombies and leaves the pid table in a consistent state.

States and exit states of a process are defined in `include/linux/sched.h` as following.

```
/* Used in tsk->state: */
#define TASK_RUNNING    0x0000
#define TASK_INTERRUPTIBLE  0x0001
#define TASK_UNINTERRUPTIBLE    0x0002
#define __TASK_STOPPED                          0x0004
#define __TASK_TRACED                          0x0008
/* Used in tsk->exit_state: */
#define EXIT_DEAD   0x0010
#define EXIT_ZOMBIE 0x0020
#define EXIT_TRACE  (EXIT_ZOMBIE | EXIT_DEAD)
```

- `TASK_RUNNING` is either a process that is ready to be run (in which case it's more like "runnable") or that is actually running.
- `TASK_INTERRUPTIBLE` and `TASK_UNINTERRUPTIBLE` are both states in which a task is sleeping, waiting for some condition to be true. The former allows a process to be woken up by signals, the latter does not: an uninterruptible task will ignore any signal and will only wake up on his condition. This distinction is the reason why, the routine that wakes up tasks is called `try_to_wake_up()`.
- `__TASK_TRACED` means that another process is tracing this one, usually a debugger such as `ftrace`.
- A task in `__TASK_STOPPED` is not running and cannot be scheduled: this happens upon stop signals or any signal from a tracing process.

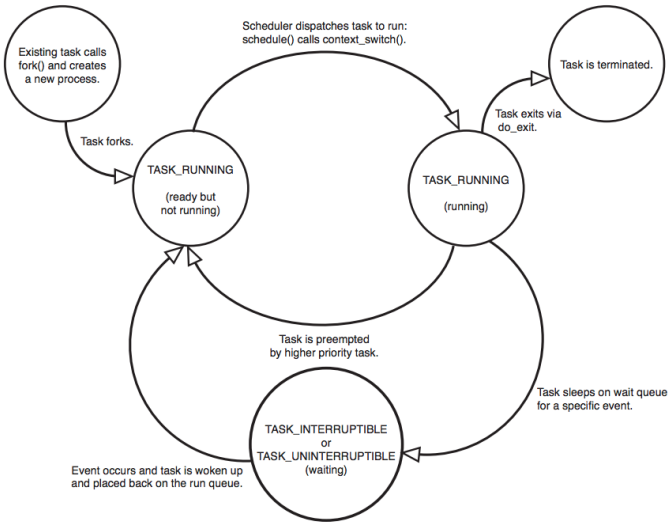


Figure 1.7: State machine of task states

Chapter 2

The Linux scheduler

As illustrated earlier, the CPU can only execute one task at a time and, even on multiprocessors systems, the number of tasks will be larger than the number of cores. For this reason, the scheduler is in charge of alternating the execution of tasks. In order to decide which task to run and for how long, the scheduler has to consider many factors, such as the importance of the task and its type.

2.1 Elements of scheduling decisions

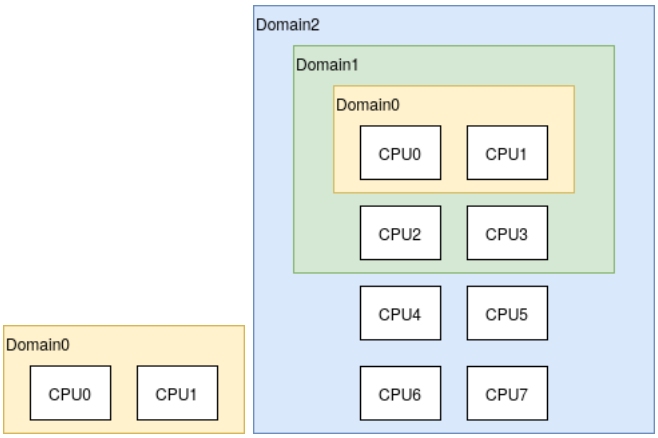
2.1.1 Domains

The domain struct is a key component in the linux scheduler. Each domain keeps two main elements: a spans array and a sched_group circular list. Above each CPU is built a hierarchy of domains, which cluster increasingly more cores and manages the related balance.

Spans Each domain clusters a cores' quantity, growing according to its level in the hierarchy. A domain's span needs to include his child's span and can extends it. The balance doesn't take place between all the CPUs' ready-queue but among the sched_groups.

Sched groups Sched_groups of a domain are partitions of its span, so each core in the span is in one and only one group. The balance is performed among them, involving the more busy and the less one of each group and balancing their ready-queue.

Hierarchy At the base level, the domains' hierarchy is based on a couple of close cores at least, meaning that there can't be a domain with a single core (2.1a). Above each domain is placed another one which has more cores, until a domain containing all CPUs in the system is made (2.1b).



(a) First step (b) Complete

Figure 2.1: Scheduler domains' hierarchy

Something often omitted is that the hierarchy shown in 2.1b can be either the CPU0 or CPU1 one, but not both: each core owns a private domains hierarchy. This implies that even if CPU0 and CPU1 are in the same domain, named Domain0 in the picture, they hold two different actual structs that represent it. This behavior helps to finer tune the domains balancing with the flags described in Section 2.3.2.

On the other hand, the groups involved in the domains structs are the same across all the system. Let's take as an example the same hierarchy shown in fig 2.1b and the groups shown in 2.2. Group0 and Group1, references of CPU0's first domain, are the same in CPU1's counterpart and a similar situation happens with the second domain of the first 4 cores: they all share the same Group2 and Group3 pointers. In this way all the domains are coherent and are able to balance processes load among the same groups.

2.1.2 Classes and policies

Each class has its code, implementing its own algorithm. Within each class, policies represent special scheduling decisions, and each process has a policy associated with it. This means that at specific points of the code the behavior may change depending on the process' policy. This may seem confusing, but consider `SCHED_RR` and `SCHED_FIFO`: they use the same priority scale, so processes of both policies share the same run queues. Furthermore, FIFO and round-robin are very similar algorithms, differing just because of round-robin having time quanta. From this perspective, it makes sense that tasks of both policies are scheduled using the same code. Scheduling classes are mapped to their code as follows:

- `dl_sched_class` – `kernel/sched/deadline.c`

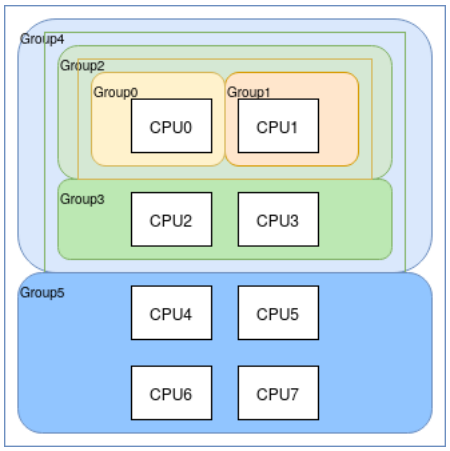


Figure 2.2: A complete domains' hierarchy

- `rt_sched_class` – `kernel/sched/rt.c`
- `fair_sched_class` – `kernel/sched/fair.c`

`SCHED_DEADLINE` is the highest priority policy. A task with this policy defines a deadline, before which the task has to finish its execution. The scheduler guarantees that the deadline is respected. To do that, when the policy or the attributes are changed, the scheduler checks the feasibility of the change, and if the CPU is too busy then it returns an error. `SCHED_FIFO` is a cooperative policy, first come first served: the tasks are executed in the order of arrival, and there is no notion of timeslice. Even if FIFO is cooperative in principle, `SCHED_FIFO` allows preemption. In fact, all scheduling on Linux is preemptive. A running task with this policy can be preempted if another task with higher priority becomes runnable. `SCHED_RR` is a round-robin policy. It's similar to `SCHED_FIFO` but it uses round-robin to cycle trough all the tasks with the same priority. Each task can run only for a maximum fixed timeslice (quantum of time) then it's preempted and put at the end of list of its priority. As for `SCHED_FIFO`, if a higher priority task becomes runnable, the current task is preempted. `SCHED_NORMAL` is the default policy that is used for regular tasks and uses CFS (the Completely Fair Scheduler, implemented in `fair.c`), this chapter later describes how it works. Most tasks run with this priority on most systems. `SCHED_BATCH` is similar to `SCHED_NORMAL` but it will preempt less frequently, so every process will run longer. For this reason, it is more suited for non-interactive workloads, typically on servers. `SCHED_IDLE` is for tasks with very low priority, almost any task can preempt tasks with this policy.

2.1.3 Priorities

Each task has a priority associated with it. The first priority levels corresponds to real-time tasks, which are scheduled with the `SCHED_FIFO` or `SCHED_RR` pol-

icy. Normal tasks are scheduled with the `SCHED_NORMAL`, `SCHED_BATCH` and `SCHED_IDLE`. These have a priority called *nice* that ranges from -20 to 19, -20 being the highest priority and $+19$ the lowest. The reason why it's called "nice" is that it represents how nice the process is being to all the others: being very nice (positive nice) means having a low priority.

The behavior of different nice values depends on the version of the scheduler. This has been a problem for Linux for years, because nice's behavior used to advantage servers over desktop[1].

2.2 Multiprocessing

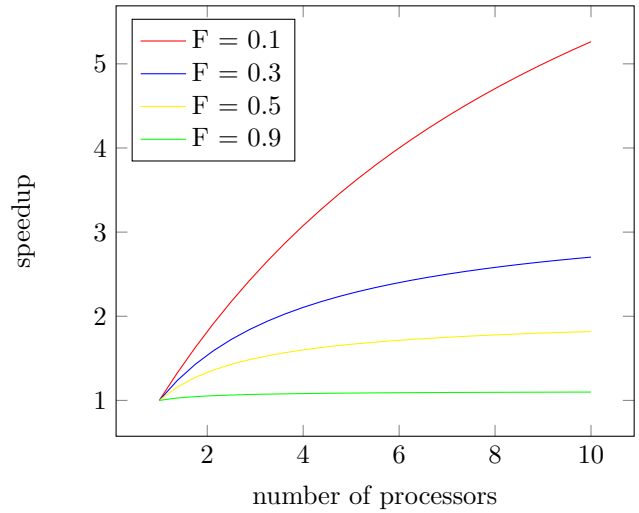
As mentioned in Chapter 1, nowadays most systems have more than one processor. Over the years, the frequencies of processors have been increased in order to achieve better performances, but there are physical limitations to this. Once we hit the limit in CPU frequency, the best way to improve the performance of a system is to add more processors. This allows us to run more processes in parallel and improve the performance, but processes often share resources together and need to communicate with each other (IPC): this factor limits the performance gained by parallel execution.

Theoretical performance gain

The performance gained with a multiprocessing system depends on the degree of parallelism of the processes: a process that has to wait for the result of another computation cannot be parallelized. Gene Amdahal's law [6] predicts the maximum theoretical improvement of multiprocessing:

$$speedup = \frac{1}{F + \frac{1-F}{N}} \tag{2.1}$$

where F is a factor that represents the portion of the calculation that cannot be parallelized and N is the number of processors.



The graph shows how the performance gained by adding processors depends on the type of the computation. A computation that cannot be parallelized much (high value of F) will have a low performance increase with more processors.

2.2.1 Load balancing

Let's consider a single CPU with multiple cores. To achieve the best possible performance, the workload should be spread evenly between the cores, and there shouldn't be any core that does significantly less work than another. It is up to the scheduler to keep the load of each core balanced.

How is the load measured? The first option is to use the weights of the task. The load of a core would be the sum of the weights of its tasks, but this approach doesn't work very well. Suppose that we have 4 tasks with the same priority, 2 are CPU intensive and 2 spend most of their time sleeping, and we want to balance the load between 2 processors. If we use only the weights of the tasks to balance, both the CPU intensive tasks could get assigned to one core, making it always busy, while the other core would be almost always idle as both of his tasks spend most of their time waiting. This approach doesn't take into consideration the nature of the tasks (CPU bound vs I/O bound). To effectively measure the load, we need to keep track of the amount of time that a task spends sleeping. Hence, the load of a task becomes a combination of its weight and its average CPU utilization, and the load of the core is the sum of the loads of its tasks. Since the CPU utilization of a task can vary, its load is constantly updated.

The load then is used as a reference to take important decisions in order to try using the computational resources in the most effective way. This decisions are taken by the load-balancer, in practice represented by the scheduler which must decide every time on which CPU the task should be executed on. Since the

definition of a balanced system may vary according to the needs, these decisions must also be tuned accordingly, and to do so, the load balancer may slightly adjust the load value for the evaluation phase. This will be explained more in depth in the section about the attribute responsible for this evaluation 2.3.1.

Migrating tasks

The scheduler periodically checks that the load of all the CPUs is balanced. In case it is not, the scheduler tries to migrate the tasks from one CPU to another: this operation is called *task migration*. The migration of a task could be expensive and, in some cases, it could be more efficient to not move the task at all. The memory access design of the CPU plays an important role and can change radically how expensive it is to migrate tasks. In a *uniform memory access* (UMA) architecture there is one memory, and the cost of accessing it is the same for all the cores. In this case the tasks can be migrated between cores without any constraints, but having a single memory is not efficient since two cores cannot access the memory at the same time. Modern processors solve this problem by giving each core its own memory, but this creates a more complex structure. In *non-uniform memory access* (NUMA) the cost of accessing the memory is no longer the same for all the cores, but depends on where the desired data is located. Specifically, memory can be local or foreign relatively to a core, and the migration cost changes drastically between the two. This means that sometimes it is more efficient to keep a task on the same core, even if the cores are not balanced.

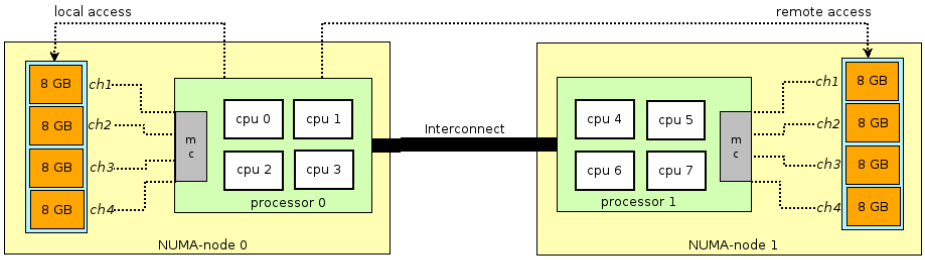


Figure 2.3: Local and foreign memory in a NUMA architecture

The same system could use a mix of the two structures: the cores could be divided in groups with their own memory. Accessing the memory of another group is more expensive, but inside a group all the cores can access the memory at same cost. Inside a group tasks can be migrated more frequently, while migration between groups should be less frequent. To efficiently use this structure the system gives the possibility of defining different migration policies.

2.3 Scheduling domains

As introduced in Section 2.1.1, the scheduling domains play a crucial role when trying to balance the different loads among the CPUs. They allow a finer control on scheduling decisions over the whole topology, making possible to adopt different strategies or policies for different groups of cores.

2.3.1 Load-balancing threshold

There is an attribute inside every scheduling domain struct called `imbalance_pct`, which is a number representing loosely speaking "how imbalanced (always in terms of load) a domain is allowed to be". To be more precise this value is used as a threshold for many cases, but primarily to decide whether a balancing operation is needed or not. Let's see in practice how changes in this value affect the domain balancing from the fair `sched_class` by looking at the code of two functions (developers comments have been left for more clarity):

```
/*
* group_has_capacity returns true if the group has spare capacity that could
* be used by some tasks.
* We consider that a group has spare capacity if the  * number of task is
* smaller than the number of CPUs or if the utilization is lower than the
* available capacity for CFS tasks.
* For the latter, we use a threshold to stabilize the state, to take into
* account the variance of the tasks' load and to return true if the available
* capacity is meaningful for the load balancer.
* As an example, an available capacity of 1% can appear but it doesn't make
* any benefit for the load balance.
*/
static inline bool
group_has_capacity(struct lb_env *env, struct sg_lb_stats *sgs)
{
        if (sgs->sum_nr_running < sgs->group_weight)
        return true;

        if ((sgs->group_capacity * 100) >
        (sgs->group_util * env->sd->imbalance_pct))
        return true;

        return false;
}

/*
*  group_is_overloaded returns true if the group has more tasks than it can
*  handle.
*  group_is_overloaded is not equals to !group_has_capacity because a group
*  with the exact right number of tasks, has no more spare capacity but is not
*  overloaded so both group_has_capacity and group_is_overloaded return
*  false.
*/
static inline bool
group_is_overloaded(struct lb_env *env, struct sg_lb_stats *sgs)
{
        if (sgs->sum_nr_running <= sgs->group_weight)
```

```
        return false;

        if ((sgs->group_capacity * 100) <
        (sgs->group_util * env->sd->imbalance_pct))
        return true;

        return false;
}
```

Both functions describe in some way the capacity of a group, which is done by checking if the *effective* group capacity exceeds the `group_util`, an attribute that can be viewed as the current load of the group, multiplied by the `imbalance_pct`; then they return their respective information to the load balancer (in the fair `sched_class` represented by `select_rq_fair()`) which will decide how to best perform the balancing. We can see that the `imbalance_pct` can be used to leverage the current load, ultimately directly affecting what to make the function return. If, for instance, we set this value to 0, the function `group_has_capacity()` will always (except when the capacity is 0 itself) return true, therefore indicating that the group allows can receive more tasks resulting in a much more frequent activation of the balancer, since the system is told that there's a group which always has capacity (which can be expressed as underloaded). If instead, we were to increase the value, a higher value of capacity will be needed in order to return true. The same reasoning can be applied in the `group_is_overloaded()` function. To summarize, the function realized by the attribute `imbalance_pct` is that of a threshold (which could be seen as percentage) indicating how tolerant to be, relative to the load of a group/cpu, before activating load balancing mechanism. Higher values are more lenient and activate the balancer less often, while lower values are more strict and result in more frequent activations. The default value is set to 125, but by activating some of the flags described in the next sections, this value changes in order to adjust for different environments.

Over the course of the versions, this threshold has been used for more than its simple and original purpose of scheduling activator, and took over some responsibility for power management, specifically in arm specializations of the kernel.

2.3.2 Flags

To accomplish this task, they keep some statistics of the processes being run on the domain, and the options on how to behave when certain events occur. This options are encoded through the use of flags, which are illustrated in the following piece of code from *topology.c*:

```
#define SD_LOAD_BALANCE                 0x0001        /* Do load balancing on this domain. */
#define SD_BALANCE_NEWIDLE        0x0002        /* Balance when about to become idle */
#define SD_BALANCE_EXEC                 0x0004        /* Balance on exec */
#define SD_BALANCE_FORK                 0x0008        /* Balance on fork, clone */
#define SD_BALANCE_WAKE                 0x0010  /* Balance on wakeup */
#define SD_WAKE_AFFINE                 0x0020        /* Wake task to waking CPU */
#define SD_ASYM_CPUCAPACITY        0x0040  /* Domain members have different CPU capacities */
#define SD_SHARE_CPUCAPACITY        0x0080        /* Domain members share CPU capacity */
#define SD_SHARE_POWERDOMAIN        0x0100        /* Domain members share power domain */
#define SD_SHARE_PKG_RESOURCES        0x0200        /* Domain members share CPU pkg resources */
#define SD_SERIALIZE                0x0400        /* Only a single load balancing instance */
#define SD_ASYM_PACKING                 0x0800  /* Place busy groups earlier in the domain */
#define SD_PREFER_SIBLING        0x1000        /* Prefer to place tasks in a sibling domain */
#define SD_OVERLAP                0x2000        /* sched_domains of this level overlap */
#define SD_NUMA                         0x4000        /* cross-node balancing */
```

This flags are set during the initialization of a SD, according to the underlying topology, or, if its definition is missing or some errors occurred, following default values. They each have their own function, but we can divide them into two general categories: those flags which enable or disable some sort of function, usually the load balancing between domains, and those which change values in order to achieve a different behavior of the system, like for instance the threshold used for indicating the load of a CPU and therefore whether a balancing is due or not. This options allow to fine tune the load balancing mechanism which can be useful when coupled with other sections of the kernel such as the power management aspect.

2.4 Objectives of the scheduler

2.4.1 Different workloads

Linux is used in all sorts of machines, from desktop computers to high-end servers to mobile devices. For this reason, the workload can vary a lot. Ideally, the scheduler should have good results in every scenario with every different workload, but in practice, this is really hard to implement. Even a small tweak to the scheduler could advantage desktop users over server users or viceversa: in other words, it's difficult to make every user happy [1]. Nonetheless, Linux

Scheduling classes	Scheduling policies
stop_sched_class	
dl_sched_class	SCHED_DEADLINE
rt_sched_class	SCHED_FIFO SCHED_RR
fair_sched_class	SCHED_NORMAL SCHED_BATCH SCHED_IDLE
idle_sched_class	

Table 2.1: Scheduling classes and policies in Linux

tries to achieve flexibility by implementing scheduling classes. Ingo Molnár, the creator of the modern Linux scheduler, writes in the patch notes "[...Scheduling classes are] an extensible hierarchy of scheduler modules. These modules encapsulate scheduling policy details and are handled by the scheduler core without the core code assuming about them too much."[20]. A task from a scheduling class can be chosen to run only if there are no runnable tasks in classes that are higher in the hierarchy. The scheduling classes are organized as shown in Table 2.1, from high to low priority.

The scheduler cannot simply choose the tasks in any order: there are some tasks that are highly time-sensitive and need to be executed as fast as possible, and others that can wait longer without any consequence. This *interactiveness* criteria is one of the most important aspects that the scheduler has to take into account.

Response time and throughput When interacting with the system, it's expected that it will react almost immediately. Clearly, the user doesn't want to wait a second (or more) between the pressure of a button and the response from the system. Hence, the scheduler aims at detecting if the process interacts with the user and tries to minimize the response time of such processes.

There are different types of tasks: a text editor, for example, will spend most of its time waiting for user input, and when the input arrives, the task should respond as fast as possible. We call this kind of task *interactive*. Other types of tasks, on the other hand, could utilize the CPU all time without ever sleeping. This means that the scheduler not only needs to minimize the response time, but it should also give precedence to tasks that are highly interactive—especially on desktop systems.

Ideally, the scheduler should do as few process switches as possible. Preempting a task, choosing another one and then context switching is a long operation. Every time the scheduler is performing a context switch, the CPU is not being used by any task. This is why, on the long run, frequent context switches will have an impact on the system performance. More specifically, they will decrease the *throughput*, which we define as the total amount of work completed per unit of time. It's important to note that we are talking about work done for the

user: switching processes doesn't count as work since it's essentially overhead. Increasing the frequency of context switches would increase the responsiveness of the system, but it would also reduce the performance. The scheduler strives to find a balance between responsiveness and performance.

Fairness and Starvation Another property that needs to be achieved is *fairness*. What it means is that any two tasks with the same priority should run for roughly the same amount of time. Indeed, a task with higher priority should have the precedence over a task with a lower one, but at the same time, every task should get at least some CPU time. A process that doesn't get any CPU time is said to *starve*. In practice, this usually happens when a process with high priority monopolizes a CPU, starving all the other processes waiting for their turn.

2.5 Prior to CFS

The *Completely Fair Scheduler* (CFS) is the default scheduler of Linux since the 2.6.23 release, as a replacement for the $O(1)$ scheduler. To understand the advantages of CFS, it is important to understand how the previous scheduler works.

2.5.1 The $O(1)$ Scheduler

In the $O(1)$ scheduler, runnable tasks are stored in two priority queues (*run queues*): an active and an expired queue. Initially, all the tasks are inside the active queue. Then, the tasks run on the CPU, in order of priority, for the assigned timeslice: when it expires, the task is moved into the expired queue. Once all the tasks have finished their timeslice, the two queues swap roles and the process starts again.

Prior to the introduction of the O(1) scheduler, there was another version of the scheduler that was much simpler. It worked well with a small number of tasks, but there were performance issues when working with many tasks. When the $O(1)$ scheduler was introduced its goal was to keep all the positive aspects of the previous scheduler, like good interactive performance and fairness, but improving the scalability. That is, improving the performance when dealing with a large number of tasks. This was achieved using only algorithms with constant complexity $O(1)$; meaning that the $O(1)$ scheduler does not have to traverse all the list of runnable tasks when deciding which one to run. Instead, this decision always takes a constant amount of time, regardless of the number of tasks.

Priority Each user task has a priority which consists of a static and a bonus priority. The static priority, which corresponds to the nice value, can have a value from -20 to 19. The bonus ranges from -5 to $+5$ and it is determined by the interactiveness of the task: if a task spends a large amount of time sleeping,

the scheduler will boost its priority, on the other hand, if a task doesn't sleep much, it will get penalized. The static priority and the bonus are then added to determine the dynamic priority, which is the priority looked by the scheduler when choosing the next process to run.

Timeslice The $O(1)$ scheduler is based on the concept of timeslice, this is the amount of time that a task runs on the CPU during a cycle. A new cycle begins when every task has been moved to the expired queue. Each priority level is mapped to a different timeslice: higher levels get mapped to more time while lower levels get mapped to less time. The timeslice can be expressed as a time quantity or as a percentage of the total time of a cycle.

Interactive tasks and heuristics Dynamic priority bonuses and penalties are based on interactivity heuristics. This heuristic is implemented by keeping track of how long a task goes to sleep [18]. A process that is marked as interactive (`SCHED_NORMAL` rather than `SCHED_BATCH`) will be reinserted in the active queue after it has expired its current timeslice. This is done to improve the response time of the system. A task is also marked as interactive depending on its dynamic priority and its nice value. It is easier for a task with a higher priority to become interactive compared to task with a lower one. A task with a nice value of -20 is marked as interactive even if it has a dynamic priority of $+3$, while a task with a nice value of 0 has to have a dynamic priority of -2. With a nice level of $+19$ it is impossible for a task to be marked as interactive, independently of the dynamic priority.

This approach is the biggest weakness of the O(1) scheduler: it generated unpredictable behavior and could cause some tasks to be marked as interactive even when they were not. Furthermore, the different kinds of complex heuristics made the code bigger and harder to understand and maintain.

Nice levels Prior to the introduction of the $O(1)$ scheduler, tasks with a nice of 19 were using too much CPU and users were complaining.[1] Because of this, $O(1)$ was specifically designed to have the minimum timeslice set to one *jiffy*, which is the minimum measurable amount of time. The amount of time corresponding to one jiffy depends on the tick rate. In the kernel, the tick rate is the constant `HZ`, which is defined as the number of ticks per second. Despite the name, this value is not tied to the processor's frequency. With the advancements of computer hardware, this value increased, and today it's usually 1000 on desktop computers. With such a value, a jiffy is equivalent to 1 msec, meaning that a task with nice $+19$ would get a timeslice of only 1 msec (only 0.1% CPU time) and this would cause too frequent rescheduling, causing problems like cache thrashing. The value of the minimum timeslice can be adjusted, but it is still `HZ` dependent.

Another problem, that can be noticed in the graph of Figure 2.4, is that the behavior of the nice level depends on its absolute value. But the nice API only allows the nice to be changed by a relative amount. This means that with the

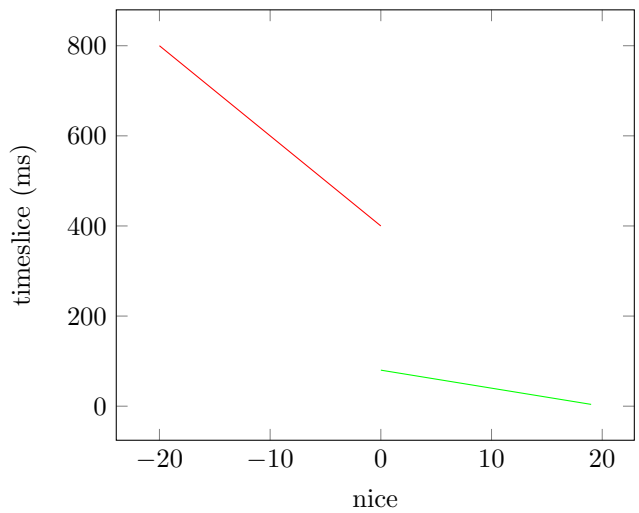


Figure 2.4: Nice to timeslice conversion

$O(1)$ scheduler, calling `nice()` to increment the nice level of a task has different effects depending on the initial value.

The last problem was that tasks with negative nice values were not responsive enough: this was problematic for multimedia applications, which had to resort to run under `SCHED_FIFO` rather than `SCHED_NORMAL`. This approach caused another major problem because `SCHED_FIFO`, as we stated earlier, is not starvation proof.

2.6 Completely Fair Scheduler (CFS)

Since version 6.6, the implementation of the default scheduler changed to Earliest Eligible Virtual Deadline First (EEVDF) [24]. In this algorithm, each task is assigned a weight w_i. Given the sum of weights $W = \sum_i w_i$, EEVDF assigns deadlines to task i according to

$$\begin{cases} d_i^{(0)} & = \text{first release time of the task} \\ d_i^{(k+1)} & = d_i^{(k)} + \frac{Q}{w_i} W. \end{cases} \tag{2.2}$$

Then the scheduler prioritizes the tasks based on the current deadline d_i. The quantum Q is the duration of the time slot a task is allocated.

2.6.1 Weight function

According to the comments in the code of `kernel/sched/core.c`, and the kernel documentation, the weight w is roughly equivalent to

$$w(n) = \frac{1024}{(1.25)^n}. \tag{2.3}$$

with n being the nice of the task, and 1024 being the weight of a task with nice 0. To avoid computing this function every time it is needed, there is a pre-calculated table which maps nice values to the corresponding weight. Below, the code from `kernel/sched/core.c` is reported. In this code fragment, the developer's comments are useful to understand how this formula works.

```
/*
 * Nice levels are multiplicative, with a gentle 10% change for every
 * nice level changed. I.e. when a CPU-bound task goes from nice 0 to
 * nice 1, it will get ~10% less CPU time than another CPU-bound task
 * that remained on nice 0.
 *
 * The "10% effect" is relative and cumulative: from _any_ nice level,
 * if you go up 1 level, it's -10% CPU usage, if you go down 1 level
 * it's +10% CPU usage. (to achieve that we use a multiplier of 1.25.
 * If a task goes up by ~10% and another task goes down by ~10% then
 * the relative distance between them is ~25%.)
 */
static const int prio_to_weight[40] = {
/* -20 */ 88761, 71755, 56483, 46273, 36291,
/* -15 */ 29154, 23254, 18705, 14949, 11916,
/* -10 */ 9548, 7620, 6100, 4904, 3906,
/* -5 */ 3121, 2501, 1991, 1586, 1277,
/* 0 */ 1024, 820, 655, 526, 423,
/* 5 */ 335, 272, 215, 172, 137,
/* 10 */ 110, 87, 70, 56, 45,
/* 15 */ 36, 29, 23, 18, 15,
};
```

The expression of Eq. (2.3) is designed in such a way that an increase of 1 in the nice value roughly translates to a 10% increase in assigned fraction of CPU. For example, if there are only 2 processes with the same nice value, they will get both 50% of CPU time. If the nice of one of the processes is increased by one, then it will get 55% of CPU time while the other will get only 45%. The figure below reports the plot of the function of Equation (2.3). In the developer's comments in the code above it's said that "to achieve that we use a multiplier of 1.25". Why exactly does 1.25 cause the "10% effect"? Let's try to reverse engineer the formula in order to understand how it was built. The expression of Eq. (2.3) can be written in a more readable format as follows:

$$w(n) = \frac{2^{10}}{\left(\frac{5}{4}\right)^n} \tag{2.4}$$

The fraction of CPU time (timeslice) given to a task is equal to its weight divided by the sum of all the other tasks' weights. Let's suppose we have only

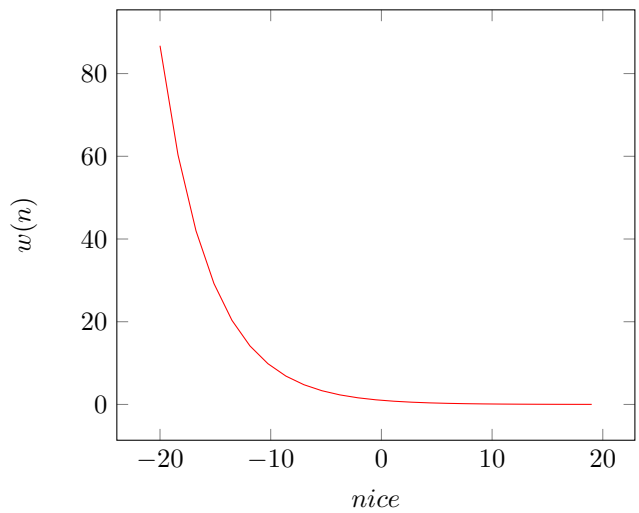


Figure 2.5: Plot of the weight as function of the nice value.

two processes and they have a difference in nice d, then the CPU percentage of the process with nice n is:

$$CPU_{\%} = \frac{w(n)}{w(n) + w(n+d)} \tag{2.5}$$

By replacing the expression for $w(n)$ of Eq. (2.4) and by setting $\alpha = \frac{5}{4}$, we find

$$CPU_{\%} = \frac{w(n)}{w(n) + w(n+d)} = \frac{\frac{2^{10}}{\alpha^n}}{\frac{2^{10}}{\alpha^n} + \frac{2^{10}}{\alpha^{n+d}}} =$$
$$= \frac{1}{\frac{\alpha^n}{2^{10}}\left(\frac{2^{10}}{\alpha^n} + \frac{2^{10}}{\alpha^{n+d}}\right)} = \frac{1}{1 + \frac{\alpha^n}{\alpha^{n+d}}} = \frac{1}{1 + \frac{1}{\alpha^d}}$$

We have now a function to calculate the $CPU_{\%}$ of a process given a difference d in nice, which is

$$CPU_{\%}(d) = \frac{1}{1 + \left(\frac{4}{5}\right)^d}. \tag{2.6}$$

This function is plotted in the figure below. Finally, by computing the derivative of Equation (2.6) at $d = 0$ we find the value of $0.5 \log(5/4) \approx 0.11157$. This is the 10% increase that was referred by the comments in line 7.

Since this value is not exactly 10%, it is possible to correct the original formula according to the precise value. With the new formula, we can compute the `prio_to_weight` table again, and then recompile the kernel with the new table: this way, the "10% effect" would be even more precise.

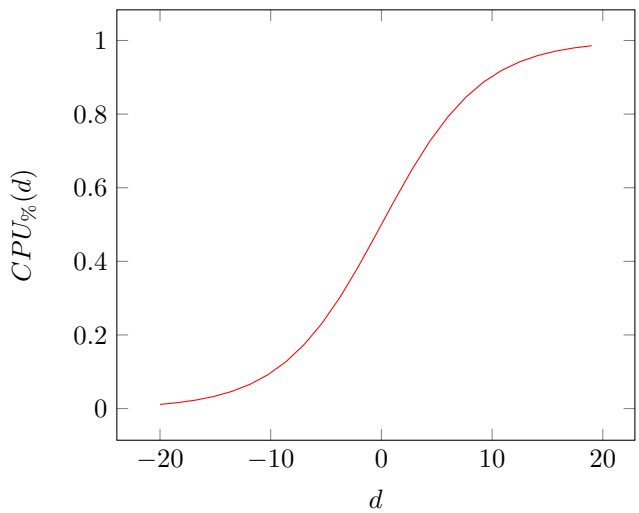


Figure 2.6: Plot of the fraction of CPU assigned to one process among two, assuming they have a difference d of the nice level.

2.6.2 Runqueue

When a task goes to sleep, it is removed from the runqueue. While sleeping, the virtual runtime of a task remains the same, while the other tasks will continue to increase their own vruntime. If the task is woken up and re-inserted into the runqueue with the same virtual runtime it had before, it would have a significantly higher priority compared to the task that remained on the runqueue: this creates a situation of unfairness, where a sleeping task gets too much priority. A similar situation happens with a newly created task, since it cannot simply have zero as the initial value. If that were the case, the older tasks would not get any CPU for a long time, and the new task would stay on the CPU until its virtual runtime reaches the one of the older tasks.

To avoid this unfairness, the scheduler keeps track of the *minimum virtual runtime* present in the run queue. That is, the virtual runtime of the most deserving active task. Every time that a task is chosen for execution, the minimum is updated. When new tasks are added to the runqueue their virtual runtime is updated to keep things fair. When a sleeping task is woken up, a similar solution is applied: before the task is queued again, the scheduler checks that its virtual runtime is at least equal to the current minimum; if it is not, then it is set to the minimum and the task is inserted into the runqueue. This way the difference in virtual runtime between all the active tasks remains small and no task gets an unfair treatment. When a new task is created via `fork()` and inserted into the runqueue, it inherits the virtual runtime from the parent: this prevents the exploit where a task can take control of the CPU by continuously forking itself.

2.6.3 Runqueue internals

The runqueue is a structure that holds the information needed to make scheduling decisions, along with information on the process currently running and the ones currently waiting. Runqueues are divided per logical CPU, offering a simple way to monitor how processes evolve during their course of lifetime, along with a simpler migration process. The runqueue structure is split between a main `rq` struct[1], and substructures for various features, along with one for each scheduling class. The following is an excerpt of the `rq` struct, with non relevant fields omitted, as only needed for optional features. It contains a lock in order to modify the structure, so that the scheduler and the task switch are always internally consistent. It also contains the substructures `cfs`[2], `rt`, and `dl`, representing the runqueues specifically for the scheduling classes of CFS, RealTime, and DeadLine. It also contains direct pointers to the current task running, whether it is a process or idle, depending on the value of the field `nr_running`.

```
struct rq {
        /* runqueue lock: */
        raw_spinlock_t                __lock;

        unsigned int                nr_running;
        /***/
        struct cfs_rq                cfs;
        struct rt_rq                rt;
        struct dl_rq                dl;

#ifdef CONFIG_FAIR_GROUP_SCHED
        /* list of leaf cfs_rq on this CPU: */
        struct list_head        leaf_cfs_rq_list;
        struct list_head        *tmp_alone_branch;
#endif /* CONFIG_FAIR_GROUP_SCHED */

        struct task_struct __rcu        *curr;
        struct task_struct        *idle;
        struct task_struct        *stop;
        /***/
}
```

Below is an excerpt of the `cfs_rq` struct. It contains the fields relating to the load of the entire runqueue, along with the number of processes in running, along with keeping track of processes running on other scheduling classes. The `min_vruntime` keeps the smallest `vruntime` in the queue. Not only this is needed as a redundancy to the CFS structure explained in Section 2.6, but to make sure that newer tasks don't get more priority than needed, being initialized at the current minimum `vruntime`, rather than a zero value. Pointers to the current, next, and previous tasks to schedule are kept to provide a mechanism to calculate scheduling slice values, along with the decision on the next task to schedule. A

[1] https://elixir.bootlin.com/linux/v6.2/source/kernel/sched/sched.h#L954
[2] https://elixir.bootlin.com/linux/v6.2/source/kernel/sched/sched.h#L550

load average is kept updated, for balancing purposes, along with fields keeping track of the entire load per scheduling group.

```
struct cfs_rq {
        struct load_weight        load;
        unsigned int                nr_running;
        unsigned int                h_nr_running;      /*
        ↪  SCHED_{NORMAL,BATCH,IDLE} */
        unsigned int                idle_nr_running;   /* SCHED_IDLE */
        unsigned int                idle_h_nr_running; /* SCHED_IDLE */

        u64                        exec_clock;
        u64                        min_vruntime;
        /***/
        /*
         * 'curr' points to currently running entity on this cfs_rq.
         * It is set to NULL otherwise (i.e when none are currently running).
         */
        struct sched_entity        *curr;
        struct sched_entity        *next;
        struct sched_entity        *last;
        struct sched_entity        *skip;

#ifdef CONFIG_SMP
        /*
         * CFS load tracking
         */
        struct sched_avg        avg;
#ifndef CONFIG_64BIT
        u64                        last_update_time_copy;
#endif
        struct {
                raw_spinlock_t        lock ____cacheline_aligned;
                int                nr;
                unsigned long        load_avg;
                unsigned long        util_avg;
                unsigned long        runnable_avg;
        } removed;

#ifdef CONFIG_FAIR_GROUP_SCHED
        unsigned long                tg_load_avg_contrib;
        long                        propagate;
        long                        prop_runnable_sum;

        /*
         *   h_load = weight * f(tg)
         *
         * Where f(tg) is the recursive weight fraction assigned to
         * this group.
         */
        unsigned long                h_load;
        u64                        last_h_load_update;
        struct sched_entity        *h_load_next;
#endif /* CONFIG_FAIR_GROUP_SCHED */
#endif /* CONFIG_SMP */

#ifdef CONFIG_FAIR_GROUP_SCHED
```

```
        struct rq                *rq;        /* CPU runqueue to which this cfs_rq is attached */

        /*
         * leaf cfs_rqs are those that hold tasks (lowest schedulable entity in
         * a hierarchy). Non-leaf lrqs hold other higher schedulable entities
         * (like users, containers etc.)
         *
         * leaf_cfs_rq_list ties together list of leaf cfs_rq's in a CPU.
         * This list is used during load balance.
         */
        int                      on_list;
        struct list_head        leaf_cfs_rq_list;
        struct task_group        *tg;        /* group that "owns" this runqueue */

        /***/
#endif /* CONFIG_FAIR_GROUP_SCHED */
};
```

2.7 Scheduling Statistics

The following field excerpt from the `rq` struct, relates to the tasks defined in Section 1.4. It keeps an updated count of the clock as defined in Section 1.4.1, along with extra fields. One example being clock_task, which removes time spent in interrupts or stolen from additional mechanisms, like para-virtualization. Other fields are instead related to the tracking of time spent idling, in order to provide accurate Per Entity Load Tracking (PELT) measures.

```
struct rq {
  /***/
        unsigned int                clock_update_flags;
        u64                         clock;
        /* Ensure that all clocks are in the same cache line */
        u64                         clock_task ____cacheline_aligned;
        u64                         clock_pelt;
        unsigned long                lost_idle_time;
        u64                         clock_pelt_idle;
        u64                         clock_idle;
#ifndef CONFIG_64BIT
        u64                         clock_pelt_idle_copy;
        u64                         clock_idle_copy;
#endif
  /***/
}
```

The following snippet of the `rq` struct, instead serves to keep information about the general state of the runqueue. This feature is activated by the flag `sysctl_schedstats`, and serves to populate the rest of `/proc/schedstat`. Without the flag turned on, only basic information about the runqueue and

tasks is kept. These fields keep count of the times various events occurred on the processor, along with the total time spent executing tasks on the processor, without including interrupts and system procedures.

```
struct rq {
  /***/
#ifdef CONFIG_SCHEDSTATS
        /* latency stats */
        struct sched_info        rq_sched_info;
        unsigned long long        rq_cpu_time;
        /* could above be rq->cfs_rq.exec_clock + rq->rt_rq.rt_runtime ? */

        /* sys_sched_yield() stats */
        unsigned int                yld_count;

        /* schedule() stats */
        unsigned int                sched_count;
        unsigned int                sched_goidle;

        /* try_to_wake_up() stats */
        unsigned int                ttwu_count;
        unsigned int                ttwu_local;
#endif
        /***/
}
```

Chapter 3

SCHED_EXT: extending the Linux scheduler

As previously discussed each scheduling class has its code and implements its own algorithm, but there is a new kernel patch sent in the Linux Kernel mailing list to add a new extensible scheduling class, called dynamically SCHED_EXT, which can be used to load at runtime into the kernel a new scheduling policies written as eBPF programs (see Chapter 4).

The algorithm is represented as an operational structure, which contains some commons attributes such as the name, some flags,... and pointer to function which represent callbacks that are used to take scheduling decision. This structure must be implemented in eBPF which can be safely loaded into the kernel.

3.1 Current state

At the time of writing, this is only a proposed change and requires to build a custom patched kernel with SCHED_EXT to enable the use of experimental support for external scheduler; that mean that it is not at all widely spread yet. The cause is that the code is currently out-of-tree in respect to the mainline upstream Linux Kernel, but this is changing in 6.11 release cycle.

Linus Torvalds announced that he intends to merge [19] the SCHED_EXT patches even though there has been some objections by other kernel developers. Torvalds feels the code is ready enough and provides real value to the mainline Linux kernel. He wrote in kernel mailing list of 11 June 2024 [1]:

> *I've never been a huge believer in trying to make everybody happy with code that is out of tree — we're better off working together on it in-tree.*

[1] `https://lore.kernel.org/lkml/CAHk-=wg8APE61e5Ddq5mwH55Eh0ZLDV4Tr+c6_gFS7g2AxnuHQ@mail.gmail.com/`

> *And using the "in order to accept this, some other thing has to be fixed first" argument doesn't really work well either (and that has been discussed for over a decade at various maintainer summits).*
>
> *Maybe the people who have concerns about this can work on those concerns when it's in-tree.*
>
> *I'm also not a believer in the argument that has been used (multiple times) that the BPF scheduler would keep people from participating in scheduler development. I personally think the main thing that keeps people from participating is too high barriers to participation.*

So it seems like, in future version of the Linux kernel, SCHED_EXT will be present. It probably will start by being not compiled-in by default, and it will still require changing build configuration. But the upstreaming of SCHED_EXT is still a big step forward allowing more scrutiny and interest to build around SCHED_EXT, so that with increased traction and work on it, maybe in a future release it will be considered ready enough to be shipped by default.

3.1.1 Installing the patched kernel

To build patched Linux kernel with SCHED_EXT [2] it is required [2] to have (at the moment of writing):

1. `clang` 16 or higher, with clang 17 recommended
2. `pahole` 1.24 or higher
3. `rustup` (to install rust for the examples schedulers in rust)

To begin with the build, clone the repository as any other git repository:

```
git clone https://github.com/sched-ext/sched_ext.git linux
cd linux
```

After that modifying the `.config` file by adding the following line that enable SCHED_EXT support into the kernel to be built while also adding some extra debug information:

```
CONFIG_BPF=y
CONFIG_SCHED_CLASS_EXT=y
CONFIG_BPF_SYSCALL=y
CONFIG_BPF_JIT=y
CONFIG_DEBUG_INFO_BTF=y
CONFIG_BPF_JIT_ALWAYS_ON=y
CONFIG_BPF_JIT_DEFAULT_ON=y
CONFIG_PAHOLE_HAS_SPLIT_BTF=y
CONFIG_PAHOLE_HAS_BTF_TAG=y
```

[2] https://itsfoss.com/compile-linux-kernel/

After doing so it is possible to continue building the kernel as of normal [3].

This work can be avoided by using a package already built of this patched kernel version. Using any Arch based distro, it is possible to use the package offered by another arch distro, CachyOs, by simply adding the repository and installing it. To add the repository, it is required to add the following lines to the end of the `/etc/pacman.conf` file:

```
# including the cachyos repos
[cachyos]
Include = /etc/pacman.d/cachyos-mirrorlist
```

```
sudo pacman -Syu linux-sched-ext-git
```

Then, it is required to change the kernel to the newly installed one. This is done by editing `/boot/loader/entries/`*`entries-name`*`.conf` where the systemd-boot entry normally resides, so that the line containing the `linux` and `initrd` keyword are as follows:

```
# the name are the name of the newly added files in /boot directory
linux   /vmlinuz-linux-sched-ext
initrd  /initramfs-linux-sched-ext.img
```

3.1.2 Motivations

While single centralized solution in the kernel like CFS and EEVDF have shown that they can be viable for most workload and can achieving a somewhat satisfactory result in all use cases, they are not without flaw. Trying to solve some of these flaw and increasing the capability and flexibility of the Linux scheduling are the reason why SCHED_EXT is being developed [4].

Accessibility One of the core principle of this new project is to be accessible, to lower the bar of entry so that more people can start making change and test new ideas. Up until now to test a new scheduling decision it has been required to learn the inner working of the Linux kernel because the scheduler is tightly connect with other part of it. One of the object of SCHED_EXT is to abstract away most of the complex communication inside the kernel to provide a streamlined and simple API which can be used as a black box.

Fast iterations Another reason that discourage people wanting to contribute to kernel schedulers is the build and test process being difficult and especially slow. Because of that another base guiding principle, connect to the first one, is to be easy to set up and iterate on the chosen algorithm, meaning that there is a push to simplify and reduce time taken by less important part of the process

[3] For more info on how to do so refer to `https://itsfoss.com/compile-linux-kernel/`

that are not the focus so to allow easier and faster testing of new ideas. The required steps for a kernel scheduler are to set up the developer environment, to build the kernel, make changes to it and then every time these changes are to be tested it is required to rebuild and rebooting to have the new kernel loaded. Also while testing because the patch made can interfere with other vital modules, it can become unstable and even crash causing a kernel panic. Instead, using SCHED_EXT it is only required to rebuild the module that has to be loaded and their dependencies. Moreover, because the module is checked before loading it cannot conflict with the rest of the system. It is safe even if the module stop working or does not take decision in time, because the kernel can always kick this and pass control to the fallback standard scheduler. This is all happen seamlessly to the user, which can continue to work on the machine unhindered.

Quick rollouts It allows for quicker rollouts of new policies, for example in case of a temporary workaround before a stable and full mitigation for a grave issue it is mainlined in the kernel.

Userspace tooling One more advantage is that it is not bound to kernel space, and instead it allows offsetting to a user space component some decisions, which is often used to implement more complex part such as load balancing. Having a component running in user space has various benefits such as having access to floating point math while kernel space has access only to fixed point math; and having access to other tooling present in user space, such as performance tracing tools. The SCHED_EXT user space component, which is normally used to load the actual scheduler, can communicate with the kernel component using map 4.1, which are essentially memory location accessible from both the kernel space and user space. These maps can be used by the kernel component to load information that are needed by the user space component to make some decision, then this decision is communicated back using maps to the fist one which actuates it.

Language support It is also possible to write the components not only in c, but also in rust, allowing more flexibility in how to write the code.

Specific scheduler Lastly, contrary to CFS and EEVDF which are single solution that tries to schedule in every single situation, SCHED_EXT allow the creation of more specialized solution that allow to obtain maximum performance in specifics workload. These could be, for example, running a set of Virtual Machine, a common task in the cloud computing space, which can be better accommodated by a specialized scheduler that know to decrease to a minimum preemption and will allocate as much as possible every VM to a specific core or group of them reducing switching to increase responsiveness.

3.1.3 Public interest and uses

These benefits have cause some interest, especially the last one permitting to optimize the workload. The two major company that are working with SCHED_EXT are Meta (previously known as Facebook) and Google.

Meta is the most invested, in fact a Meta employee, David Vernet, has been working in official capability on SCHED_EXT for about two and half years. They were already able to optimize HHVM (HipHop Virtual Machine), an open-source virtual machine based on just-in-time compilation that serves as an execution engine for the Hack programming language, used for ads, by about 1-3% RPS (Request Per Second) and 3-6% P99 latency (the 99th percentile of the latency meaning that 99% of the requests should be faster than given latency) [26].

As a company policy, Meta requires to upstream any change to the Linux kernel before using a feature internally, while this increase the work initially, it allows moving faster to the latest kernel release and reduce maintaining burden. Because of this upstreaming SCHED_EXT is considered a priority.

The patch set that allowed the previously mentioned performance increase is the SHARED_RUNQ one for the CFS. This introduces a global FIFO, which is simple and work conserving, but works the best only on single sockets. This was born, as a direct result of the SCHED_EXT experimentation. In early test in the mainline kernel it seems to be improving performances by about 10%-23% [4]

Google instead committed with the release of SCHED_EXT v4 to rebuild ghOSt on top of it. Google ghOSt is a user space scheduling framework, which has seen strong result on optimizing search and VM scheduling [26]. Like SCHED_EXT it uses BPF but only to accelerate some code path.

3.1.4 Main issues

Even though using SCHED_EXT has numerous advantages, these are not enough to dispel all doubts about possible issues arising from adopting the use of this new technology.

Inhibition of contributions The major and more common one is that it will cause CFS/EEVDF contributions to die down and to encourage building various out-of-tree or even non-GPL scheduler. While the complaint about the license is solved by the eBPF verifier 4.2.1, which check for GPL 2.0 licensing inside the code and refuse to load the module if it doesn't find it; instead the complaint about the out-of-tree schedulers remains open to discussion. But SCHED_EXT was not built as a CFS replacement, and the problem cited doesn't mean that it is necessarily a blocking one. In fact, contributions to CFS are already pretty limited because of the high bar of entry and there exist already multiples out of tree schedulers, and possibly this innovation could even improve the situation, because it would increase experimentation allowing to find the most interesting decision and to upstream those, which is possible because these schedulers must

[4] `https://lwn.net/Articles/937837/`

be GPL while the same is not true for out of tree scheduler currently. Also people who already contribute to CFS would probably continue to do so.

Scheduler for single program One more point, connected to the previous one, is the creation of scheduler for single program, for example a game with its own scheduler to maximize performance. This would cause program to require a specific scheduler to be loaded, meaning that the loaded scheduler could change quite often if this become the norm and that two of this kind of applications could not work at the same time as it would mean having two different schedulers loaded at the same time. While it is not impossible it is still unlikely to happens because the kind of software that would benefit most from this are games which are almost always written for windows; but also because software with scheduler bundled which run at kernel level are likely to face opposition in the Linux community.

User-space API Another common objection to the upstreaming is that it will impose UAPIs (User-space Application Program Interfaces) restriction on the kernel. But this is not a problem because the maintainers of SCHED_EXT don't guarantee any UAPI for the core scheduling API which is based on the BPF feature struct_ops because is a kernel structure of a kernel program (written in BPF) communicating with other kernel pars. This means that is possible for breaking changes to be merged at any time, which could cause out-of-tree scheduler to stop working.

Kernel tainting The last question to ponder, is if it will be required to taint the kernel if it has SCHED_EXT scheduler loaded. This would be done to avoid having SCHED_EXT schedulers problems reported to the kernel, increasing the burden. While it could be not required in case of a crash because when crash happen in a SCHED_EXT scheduler there is probably something interesting to look in the upstream kernel, but the same cannot be said for performance regression which could be caused solely by an out-of-tree scheduler misbehaving and which cannot be fixed upstream. While all agrees that the scheduler being used will be written in the logs, there isn't any consent on tainting or not yet, even though it probably shouldn't be treated that much differently from other eBPF tools already present in the kernel which do not taint the kernel.

3.2 SCX Repository

The SCX GitHub repository contains implementation of example scheduler, tooling and common code to simplify the creation of new schedulers.

The example schedulers contain c and rust implementations, some of which could be usable in production while others are just for illustration purposes. There are also some scheduler that make use of the possibility of a user space component such as `scx_rustland` 5.2.1 of Andrea Righi, a Canonical's employee.

3.3 Co-existence with other scheduling classes

By default, loaded sched_ext scheduler will only manage tasks in the SCHED_EXT scheduling class while the one in SCHED_NORMAL, SCHED_BATCH and SCHED_IDLE tasks will continue to be scheduled by the kernel scheduler (EEVDF).

A list of all the different scheduling class can be seen here:

```
SCHED_NORMAL = 0
SCHED_FIFO = 1
SCHED_RR = 2
SCHED_BATCH = 3
/* "SCHED_ISO": reserved but not implemented yet */
SCHED_IDLE = 5
SCHED_DEADLINE = 6
SCHED_EXT = 7
```

A task can explicitly set its own scheduling policy or can be set by another process to SCHED_EXT, which can be done without privileges because it is moving a task from a higher priority class (such as SCHED_NORMAL) to a lower one (SCHED_EXT). A task with its scheduling policy set to SCHED_EXT will be treated as SCHED_NORMAL and scheduled by EEVDF until a BPF scheduler is loaded, after which such tasks will be switched to and scheduled by such new scheduler.

A task can also be set to scheduling policy by another program by using the C function `sched_setattr(pid, attr, flags)`, with the policy specified by one of the fields of the struct `attr`.

While this allows interesting setup for testing purposes, it is not recommended running in partial mode on a production system, because of performance reason caused by the fact that the SCHED_EXT scheduler need to be scheduled by EEVDF, and that task scheduled by it can be preempted by EEVDF and other higher priority tasks. This was easily changeable in the example schedulers in the SCX repository thanks to the `-p` argument, that allowed to run the scheduler in partial mode. At the time of writing, this argument is being deprecated from the previous state of not being recommended, because it already has caused some third party benchmarks to give a wrong representation of the SCHED_EXT performance, the cause of which can be found in them running the schedulers with this argument against developers' suggestions.

It is instead recommended switching all tasks to the new loaded scheduler, which can be done by the scheduler itself by calling `scx_bpf_switch_all()` from inside its `init()` operation or by setting the SCX_OPS_SWITCH_PARTIAL flags (flag in this contest, are a bit mask, used to set or get the status of a single boolean option; in this case it has value `1LLU << 3`) in the operational structure 4.3.1. In this case, all SCHED_NORMAL, SCHED_BATCH, SCHED_IDLE and SCHED_EXT (the ones below real-time priority) tasks are all scheduled by it. Terminating the external scheduler's program, by triggering SysRq-S, or detection of any internal error (including stalled runnable tasks) aborts the BPF scheduler and reverts all tasks back to CFS.

Chapter 4

Loading kernel code at runtime with eBPF

eBPF is a safe JIT (Just In Time) that can be run in the kernel, it has a specific instruction set and encoding for such instructions. During runtime its code is read, and it is interpreted to emit instructions, as binary code, for the architecture it is run onto (x86,arm,...).

The name originally meant that it is an extended BPF where the latter was used to do custom packet filtering, with BPF being an acronym for Berkeley Packet Filter, but nowadays eBPF has a much wider and richer ecosystem of use cases and eBPF has lost its original acronym.

Historically it was required to write directly eBPF assembly, but nowadays, the LLVM (which initially stood for Low Level Virtual Machine and is a compiler backend for multiple programming languages) Clang compiler has grown support for an eBPF backend that compiles C into bytecode. That mean it is possible to write C or even Rust programs, and then compile into bytecode (eBPF assembly) using the -march=bpf parameter, that can be loaded with the `bpf()` systemcall with the command BPF_LOAD_PROG, and then run by the interpreter.

Also, to simplify the writing of eBPF programs, the kernel provides the libbpf library, which includes helper functions for loading programs and creating and manipulating eBPF objects [17].

4.1 Safety and speed concerns

While writing code that is loaded into the kernel, two important aspects must be considered which are speed, as in what is the overhead added by this extra layer of translation and how much does it impact metrics such as latency; and safety, because kernel code is run in a privileged environment if it misbehaves it could cause the whole kernel to panic and bring down the entire system.

Safety eBPF programs are statically analyzed before loading by a verifier which check for any potentially unsafe operations (see Chapter 4.2.1). That means that it errs on the side of caution, stopping the loading of eBPF program that may behave correctly, that mean there are some false positive but in these case it often mean the code should be written in a safer way to begin with. This verification process should (if it there is not a bug in the implementation of this) guarantees the code to be safe, and if it cannot guarantee that then it disallows loading into the kernel.

Speed Instead, regarding speed, the use eBPF has an overhead for the translation but from the test made by the SCHED_EXT developer it seems to be very minimal, only accounting for a couple tens of a percentage less performant than the native code [27].

This is possible because unlike the original BPF's (cBPF or classical BPF) design which didn't hold up to the times, the new eBPF was designed to be aligned to modern hardware so to make the most of new technology. The new eBPF, introduced by Alexei Starovoitov, moved to 64-bit registers; increased the number of registers from two to ten; added the BPF_CALL instruction made it possible to call in-kernel functions cheaply; and, also changed its own ISA (instruction Set Architecture) so to align more to the ones of the modern processors. These changes allowed the eBPF instructions to be mapped more closely to the hardware ISA, which made possible just-in-time compilation, yielding much improved performance. The original patch that added support for eBPF in the 3.15 kernel showed that eBPF was up to four times faster than the old classic BPF (cBPF) implementation for some network filter benchmarks, and in most were at least 1.5 times faster [17].

One advantages, instead, of eBPF is the access to easy way to interact with user space using data structure called maps, real-time shared memory location which are readable and writable from both users pace and kernel space. Because an eBPF programs is run into the kernel space after being loaded, this allows the creation of a secondary component that is separated from the first while still allowing the two information sharing with the use of maps. As one shall expect, there is an overhead, but often for complex program that run only sometimes and make use of the user space this trade off may be worth it. A perfect example of such program is a load balancing which benefits from the user space ease of development, and has minor performance demerit running only every second or so. It also allows to implement a full scheduler in user space, like the one, called scx_rustland 5.2.1, implemented by Andrea Righi [1].

4.2 Layer of a SCHED_EXT scheduler

Like said before eBPF can be used for writing a scheduler that can be loaded into the Linux kernel thanks to the SCHED_EXT patch set. To be able to do

[1] `https://ubuntu.com/blog/crafting-new-linux-schedulers-with-sched-ext-rust-and-ubuntu`

so, some component and some steps are required, here listed:

1. The first required component is a program which is responsible for asking the kernel, with the use of systemcalls, to load the scheduler. It is also responsible for logging information, warning, error message, and any performance metrics it desires to measure, of the kernel component. Finally, it is also responsible for the removal of the previously loaded scheduler, when the user requires so by pressing `Ctrl+c` and so sending a SIGINT signal to this process. To accomplish all these tasks it can make use of the tooling inside the SCX framework (see third point).

2. The actual scheduler which is implemented as a `*.bpf.c` file or `*.bpf.rs` file. It contains an operational structure (described in Chapter 4.3.1) that define the scheduler, and the implementation of the callbacks used inside the operational structures. It also contains the definition of maps used to communicate with userspace.

3. The SCX framework (contained in SCX repository 3.2) which provides `scx_rustland_core`, that is a basic implementation of a scheduler used a starting point for example implementation in the SCX repository and for other scheduler one may want to implement. It also provides `scx_utils`, which, as the name implies, contains some utilities which are commonly used, permitting to avoid having to reimplement them. These utilities can be found at `scx/rust/` and in `scx/scheduler/include`. While the use of this repository is not required, because it is possible to write a custom scheduler without it, it is strongly recommended as it represents a good starting point with base template, implementations for the most used function and some example to explain how to use it. Because of this, it will be assumed the use of it in this thesis.

4. The `bpf(BPF_PROG_LOAG, union bpf_attr *attr, unsigned int size)` systemcall to load a bpf program in the kernel.

5. The eBPF verifier, that statically controls the eBPF code before loading it.

6. The eBPF runner, needed because eBPF is not a compiled language but a JIT.

7. The modified kernel, which patch the upstream kernel adding a new scheduling class, SCHED_EXT, and adding the possibility of loading an external scheduler. If the kernel weren't patched this would not be possible and the first component mentioned would just error out instead.

4.2.1 eBPF verifier

The eBPF verifier does various checks [17] to guarantee the safety of the loaded program. If it finds any problem with the code it will fail to load.

The first stage statically analyzes the code to search for not terminating path or loop. To do so it is used a depth-first search of the CFG, program's Control Flow Graph. During this search it also checks for any unreachable portion of code which is strictly prohibited

During the second stage, the verifier simulate the execution of the eBPF program instruction by instruction, while checking that the state of the virtual machine before and after each instruction is valid. The verifier doesn't need to check every possible path, as it can recognize subset of precedent states and can "prune" the current branch, skipping its simulation.

Registers, variables on the stack with uninitialized contents (those that have never been written to) cannot be read. Also, it checks that no instructions write to the read-only frame-pointer register. Finally, all pointers accesses are checked for type, alignment, and bounds violations.

The verifier, checks even to avoid unauthorized operations. In fact, when loading the program it is required to specify the eBPF program type, which restrict the kernel function that it can call. If it tries to call any other function the verifier will fail to load it.

4.3 Definition of a SCHED_EXT scheduler

4.3.1 Operational structure

Like mentioned before an external scheduler is represented and defined with the use of an operational structure, so a eBPF scheduler is a set of eBPF programs that implements the struct `sched_ext_ops`. This struct contains callbacks defined as pointer to functions, and also the scheduler name, flags and `timeout_ms`. The only actual required field is ops.name which must be a valid eBPF object name, while all the rest are optional and if they are not implemented or defined then it is used the have default implementation or value. An example definition of this struct can be observed in the following snippet, taken from `scx/rust/rustland_core` [2]:

```
SEC(".struct_ops.link")
struct sched_ext_ops rustland = {
    .select_cpu = (void *)rustland_select_cpu,
    .enqueue = (void *)rustland_enqueue,
    .dispatch = (void *)rustland_dispatch,
    .running = (void *)rustland_running,
    .stopping = (void *)rustland_stopping,
    .update_idle = (void *)rustland_update_idle,
    .set_cpumask = (void *)rustland_set_cpumask,
    .cpu_release = (void *)rustland_cpu_release,
    .init_task = (void *)rustland_init_task,
    .exit_task = (void *)rustland_exit_task,
    .init = (void *)rustland_init,
    .exit = (void *)rustland_exit,
```

[2] https://github.com/sched-ext/scx/blob/v0.1.10/rust/scx_rustland_core/assets/bpf/main.bpf.c

```
    .flags = SCX_OPS_ENQ_LAST | SCX_OPS_KEEP_BUILTIN_IDLE,
    .timeout_ms = 5000, // 5 seconds
    .name = "rustland" // scheduler name
};
```

The `timeout_ms` field is used as the number of milliseconds, that any task is allowed to not be scheduled on any CPU, before the kernel unload the scheduler. If the scheduler surpass this time, then the kernel unload the external scheduler, reverting to the standard CFS/EEVDF scheduling. This is an extra security measure to avoid task never running causing starvation of that task. Because of its importance, while this value can be modified, it must remain under 30 seconds and any greater value shall be treated as 30 seconds.

4.3.2 Dispatch queue

Dispatch queue in SCHED_EXT are queue of tasks. They can operate both as FIFO or priority queue, and, while it can manage an arbitrary number of DSQs using `scx_bpf_create_dsq()` and `scx_bpf_destroy_dsq()`, by default there are one global FIFO (SCX_DSQ_GLOBAL), and a local one per CPU (SCX_DSQ_LOCAL). These internal DSQs such cannot be priority queue.

These are created in order to match the impedance between the scheduler core and the BPF scheduler. Every CPU so has its own local DSQ from which it executes task, starting from the first one (FIFO). Every operation that it can be done on a task is represented by moving a task from a queue to another, this operation it called "dispatching to a DSQ". Dispatching to a local DSQ represent marking a task to be run on a CPU. A non-local DSQ can be "consumed", as in a task is moved to the consuming CPU's local DSQ.

The consumption from the global DSQ is done automatically, if a CPU find its local DSQ empty when searching for a task that it can run. This behavior can be change, like most of the default one using a flag.

4.3.3 Callbacks

A BPF scheduler is mostly implemented as a set of callback to be invoked via an operations structures, every of which inform the BPF of an event or a decision to be made requesting for a response.

Some of the more important callback SCHED_EXT, useful to understand and use it are here listed:

prep_enable(), enable() inform that a process is entering SCHED_EXT class, and can be used to set up associated data to the task. The first one is allowed to block and is used for memory allocations, while the second one cannot block but is the one to actually enabling scheduling for the new task.

select_cpu() select a CPU for a task that is just waking up, it should return the number of the CPU to place the task on. This decision can be used by the scheduler to wake the selected CPU if it is currently idle. This is just an optimization (if the task run on the CPU that was selected, it causes is a slight boost to performance), but can be changed again before running that task.

enqueue() Enqueue a task into the scheduler for running. Normally inside it, it is called the `scx_bpf_dispatch()` function to place a task into a chosen dispatch queue, from which it will eventually be run. But dispatching the task to be run is not required and the enqueue callback could stash the task in some structure for some time. If this is done the scheduler need to guarantee that the task is not forgotten for more than the `timeout_ms`, or it will be kicked by the watchdog.

dispatch() called on empty CPU dispatch queue to assign new task to keep CPU busy. If it remains empty after the call, the CPU will try to grab some from the global queue.

update_idle() inform the scheduler of a CPU leaving or entering idle.

runnable(), running(), stopping(), quiescent() inform the scheduler of a status changes of a task (call respectively when a task: become runnable, start running on CPU, is taken off a CPU, is no longer runnable).

cpu_acquire(), cpu_release() inform scheduler of availability change of a CPU. Example of `cpu_release` is when a real-time scheduling class claim a CPU for it to manage.

4.3.4 Functions

There are also function that can be called by a BPF scheduler to communicate scheduling decision or to use utility methods.

To communicate scheduling decision we use dispatch queue and moving tasks between them. For that scope it is possible to use functions such as the one that consume a task, called `scx_bpf_consume(task)`; or it can be used `scx_bpf_dispatch(task, queue, time_slice, enqueue_flags)` that move a flag to the requested queue applying to it a `time_slice` and some `enqueue_flags`. The time slice is the length of the maximum time a task can run on the CPU without being preempted. If, it specified as SCX_SLICE_INF, it will go into tickless mode, where it can only be preempted manually or removed by a higher priority tasks. It is also possible to instead call `scx_bpf_dispatch_vtime()` to dispatch the task on a priority queue.

An example of a utility function is `scx_bpf_kick_cpu()` can be called to preempt a task that is running on a CPU, removing it and then calling back the scheduler to pick a new task to run there.

4.4 Scheduling cycles

The following 4.1 briefly shows how a waking task is scheduled and executed.

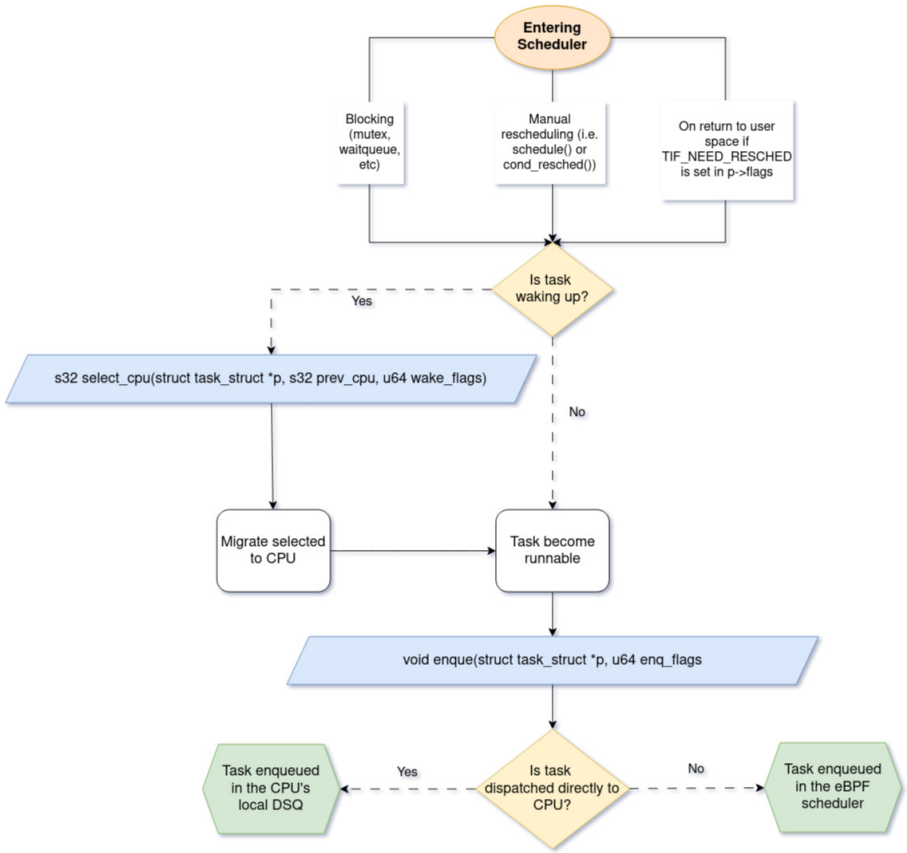


Figure 4.1: Flowchart of scheduling a task from wake [26]

ops.select_cpu() is invoked, as previously mentioned, for two reason: as a not binding optimization hint and to wake the selected core if idle. In case of Note that an invalid CPU selection will be ignored, for example if selecting a core that is outside the allowed cpumask of the task. While the scheduler can manually wake up any CPU using call `scx_bpf_kick_cpu()`, using this function can be simpler and more efficient.

A task can be immediately dispatched to a DSQ by calling `scx_bpf_dispatch()`, and doing this will cause the callback `ops.enqueue()` to be skipped.

ops.enqueue() is then invoked (if not skipped), this function has to make a decision about what to do with the task. It can dispatch it to the global or

local or even custom DSQ by calling `scx_bpf_dispatch()` using respectively SCX_DSQ_GLOBAL, SCX_DSQ_LOCAL or a DSQ ID smaller than 2^{63}; or queue the task on the BPF side.

The following 4.2, instead, display how a CPU fetch task to run:

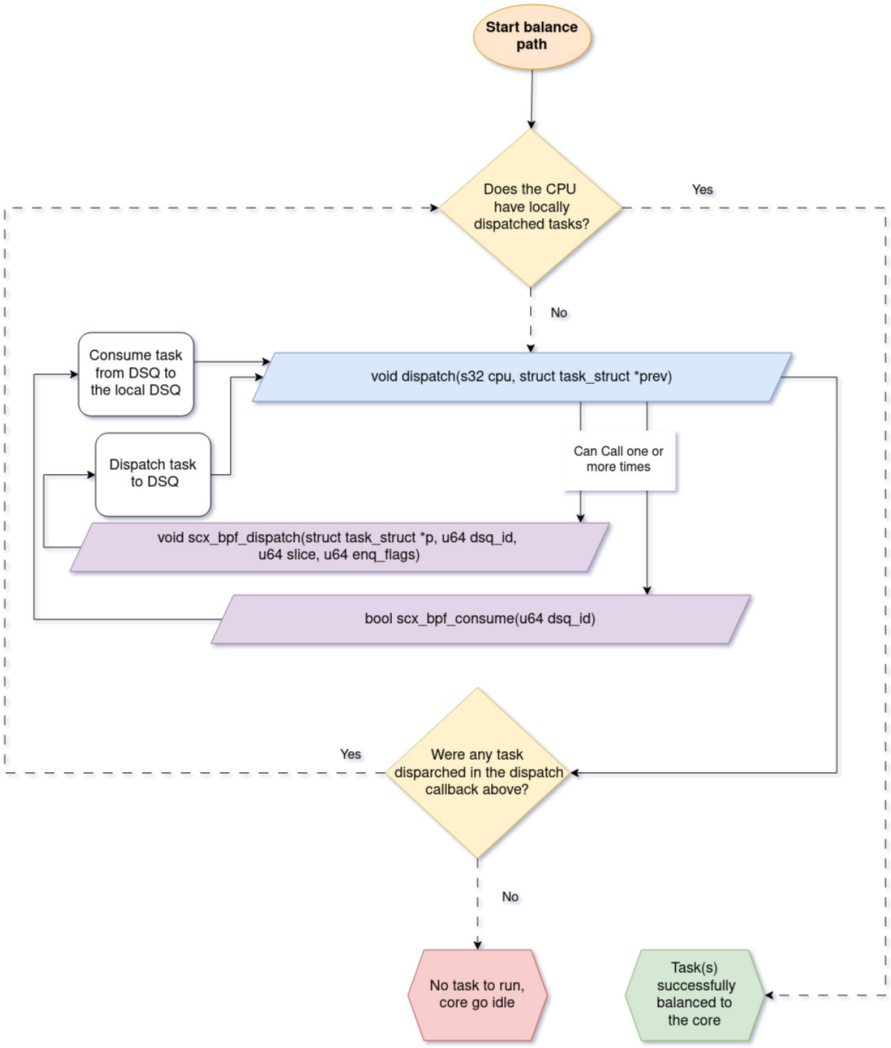


Figure 4.2: Flowchart of finding a task to run on a CPU [26]

ops.dispatch() is called when a CPU is ready, after it has search both local and then global DSQ, and it still doesn't have a task to run.

Inside it can call `scx_bpf_dispatch()` to dispatch a task to a DSQ with any possible target (such as SCX_DSQ_LOCAL, SCX_DSQ_GLOBAL or a custom DSQ), which schedules dispatching instead of immediately performing them. These tasks remain pending until a flush is required or called for explicitly; or until it is reached the maximum number of pending, `ops.dispatch_max_batch`.

Otherwise, `scx_bpf_consume()` can be called to transfer a task from the specified non-local DSQ to the dispatching DSQ. This function flushes the pending dispatched tasks before trying to consume the specified DSQ, so to avoid consuming an already dispatched task.

There are two things to notes: firstly while using only the built-in DSQs there is no need to implement this callback as a task is never queued on the eBPF scheduler and both the local and global DSQs are consumed automatically; secondly as of now both these two functions, `scx_bpf_dispatch()`, and `scx_bpf_consume()` cannot be called while holding eBPF locks, this is a known limitation, and at the time of writing it is being worked on so that in future release it will be supported.

After ops.dispatch() returns, the CPU search again for tasks in the local then global DSQ; then if `ops.dispatch()` has dispatched any tasks, retry calling it; if nothing is found it can keep executing the previous task of the CPU if it is still a runnable SCX; finally if all other fails then it will go into idle.

Chapter 5

Experimenting with SCHED_EXT

5.1 Building SCX framework

Once installed SCHED_EXT (as detailed in the section 3.1.1), it is required to also build the SCX repository[1], as a starting point. This can be done as follows:

```
# install the required dependencies
pacman -S base-devel

# clone and cd to the project directory
git clone "https://github.com/sched-ext/scx"
cd $SCX
# build the project using meson
meson setup build --prefix ~
meson compile -C build
meson install -C build
```

Note that if `sscache` is installed it is automatically used causing one of the build process to fail. To disable it and resolve the issue you can run meson with `CC=lang meson`.

5.2 Testing the example

Now, to switch to an example scheduler just run a SCHED_EXT binary (ex. `./scx_simple` while inside the build directory) and to stop it just press `Ctrl+c` on the terminal on which it was run. To see if it is enabled any SCHED_EXT kernel and which one is enabled use the following commands:

[1] `https://github.com/sched-ext/scx`

```
cat /sys/kernel/sched_ext/state /sys/kernel/sched_ext/*/ops 2>/dev/null
# This return "disabled" if no external scheduler is installed
# This return "enabled" and the scheduler name otherwise
```

Or use the python script to get more information

```
tools/sched_ext/scx_show_state.py
    ops           : simple
    enabled       : 1
    switching_all : 1
    switched_all  : 1
    enable_state  : enabled (2)
    bypass_depth  : 0
    nr_rejected   : 0
```

5.2.1 Explanation of examples present

In the SCX repository there are various examples, some productions ready while others not. Example from the first one are:

1. `scx_rusty` has a rust user space load balancer, with multi-cache awareness (L3, CCX) and that build automatically the topology
2. `scx_simple` is the simplest example that is still usable
3. `scx_flacg` which has a flattened cgroup hierarchy to reduce time to traverse hierarchy

Instead examples of not productions ready schedulers are:

1. `scx_qmap` is simple scheduler with five level FIFO queue,
2. `scx_central` has a single core that schedules for every core, deciding what to do.
   (a) reschedules sending an IPI (Inter-Processor Interrupt) to kick task
   (b) Useful for workload that requires few interruptions (ex. VMs, gaming)
3. `scx_pair` is an example of stop gap solution for L1TF[2] before core scheduling was merged
4. `scx_rustland` is an example of a user-space scheduler written in Rust, that implements the actual scheduling policy, with only the low level sched_ext functionalities implemented by scx_rustland_core, a BPF component.

[2] `https://www.kernel.org/doc/html/latest/admin-guide/hw-vuln/core-scheduling.html`

5.3 Writing custom scheduler

In this section, it will be analyzed a small but still useful example of custom scheduler which is written using `scx_simple` 5.2.1 as a starting point. This scheduler (of which the code is shown below) demonstrate the loading and termination of the scheduler, the use of the operational structure (described in Chapter 4.3.1) the use `task_struct` struct to get information about a single task, simple maps to log statistics, how to dispatch a task and what parameter can be set when doing so.

The code is split between two file `scx_simple_mine.c` which is used to log message, to load the scheduler and to handle closing operations; while the other, `scx_simple_mine.bpf.c`, contains the code that describes the actual scheduling decisions.

5.3.1 The code to load a scheduler

```
int main(int argc, char **argv)
{
    struct sigaction sa;
    sa.sa_handler = &sigint_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    sigaction(SIGINT, &sa, NULL);
    sigsigaction(SIGTERM, &sa, NULL);
```

```
static volatile int exit_req;

static void sigint_handler(int simple)
{
    exit_req = 1;
}
```

The first snippet above is from the `scx_simple_mine.c`, which from now on we will call the loader for simplicity's sake, and it contains the start of the `main` function, the entry point of this program. The first operation that it does after being executed is to set the function `sigint_handler` as the handler of the signals SIGINT and SIGTERM. The `sigint_handler` function, which is shown in the second snippet, just set a flag `exit_req` that communicate that it was required to exit the program (by pressing `Ctrl+c` for example). This flag must be set to be `volatile` to communicate that the variable should be checked every time a condition use it, instead of assuming the value hasn't changed.

```
#include "scx_simple_mine.bpf.skel.h"
```

```
    libbpf_set_strict_mode(LIBBPF_STRICT_ALL);

```

```
    struct scx_simple_mine *skel;
    skel = scx_simple_mine__open();
    SCX_BUG_ON(!skel, "Failed to open skel");
```

After the previous code, in this the loader loads the `scx_simple_mine` struct which represents a skeleton keeping together various struct that contains data, options, stats,... of the scheduler we want to load with `scx_simple_mine__open`, a function contained inside the `scx_simple_mine.bpf.skel.h` file, that is auto-generated using the scheduler name at compile time by `bpftool`. If it fails and skel is NULL then an error is thrown.

```
    __u32 opt;
    while ((opt = getopt(argc, argv, "ph")) != -1) {
        switch (opt) {
        case 'p':
            skel->rodata->switch_partial = true;
            break;
        default:
            fprintf(stderr, help_fmt, basename(argv[0]));
            return opt != 'h';
        }
    }
```

```
const char help_fmt[] =
"A simple sched_ext scheduler.\n\n"
"See the top-level comment in .bpf.c for more details.\n\n"
"Usage: %s [-f] [-p]\n\n"
"  -p            Switch only tasks on SCHED_EXT policy intead of all\n"
"  -h            Display this help and exit\n";
```

It then checks the command line options and if it finds a `-p` it switches a flag inside `skel` to signal to the scheduler component that it should run in partial mode (see Chapter 3.3). If instead it finds a `-h` it prints the help from the `help_fmt`.

```
    struct bpf_link *link;
    SCX_OPS_LOAD(skel, simple_mine_ops, scx_simple_mine, uei);
    link = SCX_OPS_ATTACH(skel, simple_mine_ops);
```

It is then loaded the operational structure, with the pointers to the functions that the scheduler use to make decisions. This structure is then attached to scheduler skeleton with a function that return a representation of the link between the two, that is used when exiting to undo this connection.

```
    while (!exit_req && !UEI_EXITED(skel, uei)) {
        __u64 stats[STAT_SIZE];

        read_stats(skel, stats);
```

```
        printf("local=%llu global=%llu little=%llu big=%llu\n", stats[0],
        ↪  stats[1], stats[2], stats[3]);
        fflush(stdout);
        sleep(1);
    }
```

This is the main loop of this program, it stays inside it until the `exit_req` become true when the user require to close the program or until `UEI_EXITED(skel, uei)` become true if the scheduler has exited for any reason (for example when failing to schedule a task for so long that, the watchdog kick the scheduler). Every second in this loop it read the stats and prints them to the screen, displaying respectevely the number of task queued locally, globally, with small time slice and with large time slice.

```
static void read_stats(struct scx_simple_mine *skel, __u64 *stats)
{
    int nr_cpus = libbpf_num_possible_cpus();
    __u64 cnts[STAT_SIZE][nr_cpus];
    __u32 idx;

    memset(stats, 0, sizeof(stats[0]) * STAT_SIZE);

    for (idx = 0; idx < STAT_SIZE; idx++) {
        int ret, cpu;

        ret = bpf_map_lookup_elem(bpf_map__fd(skel->maps.stats),
                        &idx, cnts[idx]);
        if (ret < 0) continue;
        //it is for single cpu and then added
        for (cpu = 0; cpu < nr_cpus; cpu++)
            stats[idx] += cnts[idx][cpu];
    }
}
```

This is the function that is used to read the statistics from the scheduler. It starts by getting the number of CPU and creating a buffer `cnts` which contains the info for each CPU. Then for each index it read an array representing the value on all CPU, which is then summed to obtain the total value for that information and saved in the array that is passed as parameter to this function.

```
    bpf_link__destroy(link);
    UEI_REPORT(skel, uei);
    scx_simple_mine__destroy(skel);
    return 0;
}
```

Finally, if the main loop has ended, then the closing operations start. The connection `link` is broken, it is reported the end of the run to the scheduler and the struct `skel` is destroyed and deallocated.

5.3.2 The code to define a scheduler

```
/* SPDX-License-Identifier: GPL-2.0 */
#include <scx/common.bpf.h>

char _license[] SEC("license") = "GPL";

#define SHARED_DSQ 0
#define NSEC_PER_MSEC ((u64)1e6)

const volatile bool switch_partial;

UEI_DEFINE(uei);
```

This snippet, instead from the file `scx_simple_mine.bpf.c` that is used to define how the scheduler behaves, illustrates the first lines of this file that contains file inclusion and constants declaration. But, an important thing to note is the lines that declares this file to be under GPL 2.0 license. This is required for the scheduler to be loaded as stated in Chapter 3.1.4. It then defines `uei`, which in this example will only be used to communicate the exiting of the scheduler; and a `switch_partial` constant, the value of which is automatically obtained from the `skel` struct in the loader.

```
SEC(".struct_ops.link")
struct sched_ext_ops simple_mine_ops = {
    .select_cpu = (void *)simple_mine_select_cpu,
    .enqueue    = (void *)simple_mine_enqueue,
    .dispatch   = (void *)simple_mine_dispatch,
    .init       = (void *)simple_mine_init,
    .exit       = (void *)simple_mine_exit,
    .flags      = SCX_OPS_ENQ_LAST,
    .name       = "simple_mine",
};
```

This is the core of the scheduler definition, the operational structure that define the function called at a particular event or request, plus some flags and a name. In this case only the callback here listed are defined, while the other will default back to a default implementation. In this example one flags is used to disable an optimization that will be applied automatically otherwise. SCX_OPS_ENQ_LAST [3] stop the CPU from keeping running the current task if there is no other task to run.

```
s32 BPF_STRUCT_OPS_SLEEPABLE(simple_mine_init)
{
    if (!switch_partial)
        scx_bpf_switch_all();

    return scx_bpf_create_dsq(SHARED_DSQ, -1);
```

[3] https://github.com/sched-ext/sched_ext/blob/sched_ext-v5/include/linux/sched/ext.h#L88

```
}

void BPF_STRUCT_OPS(simple_mine_exit, struct scx_exit_info *ei)
{
    UEI_RECORD(uei, ei);
}
```

These are the init function and the exit function. The first just check whether it is run in partial mode, and if it isn't, it switches all task to be managed by this scheduler; it also creates a global DSQ and return whether it successfully created this. The second one simply records that is closing when it is called. All callbacks are defined similarly using the macro BPF_STRUCT_OPS or BPF_STRUCT_OPS_SLEEPABLE with arguments the callback and the parameters that such function require.

```
s32 BPF_STRUCT_OPS(simple_mine_select_cpu, struct task_struct *p, s32
↪  prev_cpu, u64 wake_flags)
{
    bool is_idle = false;
    s32 cpu;

    cpu = scx_bpf_select_cpu_dfl(p, prev_cpu, wake_flags, &is_idle);
    if (is_idle) {
        stat_inc(0);        /* count local queueing */
        scx_bpf_dispatch(p, SCX_DSQ_LOCAL, get_time_slice(p), 0);
    }

    return cpu;
}

void BPF_STRUCT_OPS(simple_mine_enqueue, struct task_struct *p, u64 enq_flags)
{
    stat_inc(1);        /* count global queuing */

    scx_bpf_dispatch(p, SHARED_DSQ, get_time_slice(p), enq_flags);
}

void BPF_STRUCT_OPS(simple_mine_dispatch, s32 cpu, struct task_struct *prev)
{
    scx_bpf_consume(SHARED_DSQ);
}
```

In this example the `select_cpu` callback just choose a CPU using the default function `scx_bpf_select_cpu_dfl`. If this CPU is idle then it dispatches there the task, else it does nothing more. similarly, `enqueue` just dispatch the newly added task to the global queue. Both use the `stat_inc` function to increase by one of the scheduler stats; and both use `get_time_slice` to get for how long should the task run based on its priority.

Finally, the `dispatch` callback just consume a task from the global queue to the CPU that caused this callback.

```
static u64 get_time_slice(struct task_struct *task){
    // priority in standard goes with range [100, 139]
    // convert to normal form with range [-20, 19]
    const int norm_prio = task->prio - 120;

    int msecs;
    if(norm_prio <= 0){
        stat_inc(3);  /* count "big" queued */
        msecs = 20;
    }else{
        stat_inc(2);  /* count "small" queued */
        msecs = 10;
    }

    return msecs * NSEC_PER_MSEC;
}
```

The `get_time_slice` function, which is defined here above, obtain the task priority and map to the normal priority interval (for task in SCHED_NORMAL) between -20 and 19 (inclusive). Then, if the priority is low, meaning lower or equals to zero, it returns a large time slice, else it returns a tiny time slice. It also increases by one either the "small" or "big" statistics.

```
struct {
    __uint(type, BPF_MAP_TYPE_PERCPU_ARRAY);
    __uint(key_size, sizeof(u32));
    __uint(value_size, sizeof(u64));
    __uint(max_entries, 4);
    /* here used to store [local, global, big, small] */
} stats SEC(".maps");


static void stat_inc(u32 idx)
{
    u64 *cnt_p = bpf_map_lookup_elem(&stats, &idx);
    if (cnt_p) //if not null -> if is in map
        (*cnt_p)++;
}
```

Finally, this is the code that is manages the map used to communicate the statistics. It first defines a particular struct, that is recognized by eBPF, and it is used to create maps. This map in particular, is set to be a per-CPU type meaning there is a value for every single CPU, which represent the value of that information on that CPU; it has key of `unsigned int` type and value of `unsigned long` type with a maximum number of entry of four.

The `stat_inc` function, is the one that modify the value inside the map. It looks for the specific statistic, whose index is passed as parameter of the function, and if it is found, it is incremented by one.

5.4 Writing custom workload for the scheduler

To see the custom scheduler we just wrote in action we can write a simple dummy workload, which just waste clock cycles counting (in the simplest form just adding one to a variable). The following is the code, named `loop.cpp`, of this workload:

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
#include <sys/time.h>
#include <sched.h>

#define CPU_TO_RUN_ONTO 15
#define ASSERT(cond, err_str) \
    if((cond)){ perror((err_str)); exit(1); }

volatile bool running;
volatile long long int first_half_work = -1;
volatile long long int counter = 0;

void handle_signal(int sig);

void setup_shed_param(){
    // Setting itself (0) in the sched_ext class (7)
    struct sched_param par;
    sched_getparam(0,  &par);
    const int ret1 = sched_setscheduler(0, 7, &par);
    ASSERT(ret1 != -1, "sched_setscheduler");

    // Setting itself on single specific core
    cpu_set_t mask;
    CPU_ZERO(&mask);
    CPU_SET(CPU_TO_RUN_ONTO, &mask);
    const int ret2 = sched_setaffinity(0, sizeof(mask), &mask);
    ASSERT(ret2 != -1, "cpuaffinity");
}

void start_timer(int  number_millisec){
    struct sigaction sa;
    struct itimerval timer;

    // Registering signal handler
    sa.sa_handler = &handle_signal;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    const int ret1 = sigaction(SIGALRM, &sa, NULL);
    const int ret2 = sigaction(SIGUSR1, &sa, NULL);
    ASSERT((ret1 == -1 || ret2 == -1), "sigaction");

    // Setup timer
    int number_sec = number_millisec / 1000;
    number_millisec = number_millisec % 1000;
    timer.it_value.tv_sec = number_sec;
    timer.it_value.tv_usec = number_millisec * 1000L;
```

```
    timer.it_interval.tv_sec = 0;
    timer.it_interval.tv_usec = 0;

    const int ret3 = setitimer(ITIMER_REAL, &timer, NULL);
    ASSERT(ret3 != -1, "cpuaffinity");
}

int get_ms(int argc, char **argv){
    // Get timer value
    ASSERT(argc != 2, "Usage: program_name <ms_duration>");

    // Convert the string argument to a number
    int millisec;
    const int ret = sscanf(argv[1], "%d", &millisec);
    ASSERT((ret == 1 && millisec > 0),
            "First parameter should be a positive integer");

    return millisec;
}

void handle_signal(int sig) {
    if(sig == SIGUSR1)
        first_half_work = counter;
    else if (sig == SIGALRM)
        running = false;
}

int main(int argc, char **argv) {
    int ms = get_ms(argc, argv);

    setup_shed_param();
    start_timer(ms);
    running = true;

    // Busy waiting to keep the program alive
    while (running)
        counter++;

    // Print result
    printf("File %s made: %lld", argv[0], counter);
    if(first_half_work != -1){ //if was split in two times
        printf(" (%lld before and %lld after)",
                first_half_work, (counter - first_half_work));
    }
    printf("\n");
}
```

At the start of the program's execution, it gets by the program argument the length of time in milliseconds that it has to run for. It accomplishes that by setting a timer with `setitimer(ITIMER_REAL, &timer, NULL)` at the end of which the signal received by the OS is handled by the `handle_signal` function that set the running flag to false marking the end of the execution. At the start it also moves, using `sched_setscheduler(0, 7, &par)`, itself (passing 0 as pid) to the SCHED_EXT scheduling class (which is represented by the 7 as said before). That mean it is scheduled by our custom scheduler (if this is

running).

At the end of the program it prints the amount of work done during the execution, which is equals to the value in the counter used to waste clock cycles. It can also print the value before and after the receiving of (the last) SIGUSR1 signal which is used by the bash script (that follow below) to testing change of priority at runtimes.

The script below, called `double.sh`, allow running two instances of the above program, making some test playing with the priority of the process.

```bash
#!/bin/bash

# Times in mills (before quitting and before changing)
TIME_TOTAL=$1
TIME_IN_BETWEEN=$2
if [ "$TIME_IN_BETWEEN" -gt "$TIME_TOTAL" ]; then
    echo "Total time must be larger than interval before signal"
    exit
fi

if test -z "$TIME_IN_BETWEEN"; then
    # Set A to small time slice and B to big time slice
    nice -n 19 ./a.out $TIME_TOTAL &
    pid1=$!
    ./b.out $TIME_TOTAL &
    pid2=$!
    echo "STARTED NORMAL MODE. With pids: $pid1 $pid2"
else
    # Set A and B with small time slice
    ./a.out $TIME_TOTAL &
    pid1=$!
    ./b.out $TIME_TOTAL &
    pid2=$!
    echo "STARTED TWO TIME MODE. With pids: $pid1 $pid2"

    # wait for interval
    SEC_IN_BETWEEN=$(echo "scale=2; $TIME_IN_BETWEEN/1000" | bc)
    sleep $SEC_IN_BETWEEN

    # change both to big time slice
    # and ask to save workload up to first point
    renice 19 -p $pid1 > /dev/null
    kill -s SIGUSR1 $pid1
    renice 19 -p $pid2 > /dev/null
    kill -s SIGUSR1 $pid2
    echo "CHANGED PRIORITY of both from 0 to 19"
fi

# Wait for both to finish (for easier use of trace-cmd)
wait $pid1 || echo "error on pid1"
wait $pid2 || echo "error on pid2"
```

It represents two tests, one that is triggered if the TIME_IN_BETWEEN given is zero, in which case it runs the two executable (of the same `loop.cpp` file) a.out and b.out with different priority and run it for TIME_TOTAL milliseconds;

otherwise it runs both with the same priority then lowering both priority after TIME_IN_BETWEEN milliseconds.

At the end of the execution we can see the work, total and only before or after TIME_IN_BETWEEN if this is set, done by the two programs. To capture what has happened in more detail, double.sh can be passed as arguments of trace-cmd command, and it can be visualized with the use of Kernelshark 6. The result of this work are the following images.

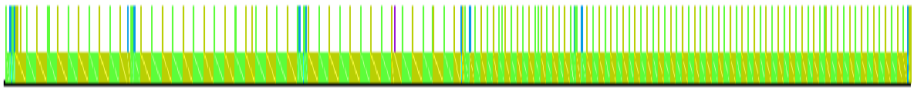


Figure 5.1: Resulting trace visualization using Kernelshark

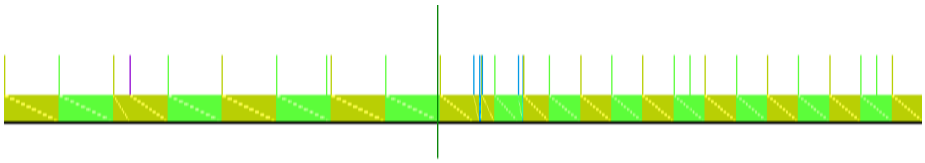


Figure 5.2: Region in which the two task's time slice changed

Chapter 6

KernelShark

6.1 Tracing

It is important to understand the difference between simple event logging and tracing. The former is often used by system administrators to catch and resolve high-level issues (e.g. failed installation of programs or intrusion detection). It must be easy, so the logs must not be "noisy". On the other hand, tracing is consumed primarily by developers and logs low-level information (e.g. thrown exception). Since it handles lower level information it is necessarily "noisy", this means that reading the logs is not always intuitive or simple.

In operating system based computing environments a significant amount of a process' behavior is defined by its interface with the operating system. This interface typically defines the process' environment such as the current directory, the input/output operations including operations of files, the execution of sub-processes and inter-process communication. Logging and interpreting the transactions between a process and the operating system it runs on, can provide data that can be used for a variety of purposes. Some of them are the following [23].

Debugging A listing of a program's operating system calls allows the programmer to analyze its behavior at a low but well-defined level. Program errors can be explained in terms of the operating system calls that were (or were not) issued, and these can in turn point to the source of the error.

Profiling System calls can consume a significant amount of program run time, since the state of the machine must be saved and restored between calls. A listing of the system calls can provide hints on areas of a program that can be optimized to enhance a program's speed.

Program verification A log of a program's transactions with the operating system can be used to verify a program against its specifications or a run of

a previous version. In addition, the log can be used to detect the use of non-portable functions, or programs that have been infected by viruses.

In other words, tracing is mainly used to understand what is happening and how a given system behaves meanwhile performing a certain operation. To do tracing you must always, for better or worse, be able to intercept some operations and make sure that a "trace" remains (from here the name) somewhere (e.g. file, buffer in memory). What operations are traced and how they actually do so define the tracing mechanism itself. But is tracing really useful? Computer systems, both at the hardware and software-levels, are becoming increasingly complex. In the case of Linux, used in a large range of applications, from small embedded devices to high-end servers, the size of the operating system kernels increases, libraries are added, and major software redesign is required to benefit from multicore architectures, which are found everywhere. As a result, the software development industry and individual developers are facing problems which resolution requires to understand the interaction between applications and all components of an operating system [15].

6.1.1 Event tracing with trace-cmd

While function tracing (tracing which function an in which order are called) is very useful and will come in handy to understand the code, in this chapter we we will focus on event tracing, especially using `trace-cmd`.

```
lorenzo@localhost:/HDD/tesi/dat> sudo trace-cmd record -e kvm:kvm_exit
Hit Ctrl^C to stop recording
^CCPU0 data recorded at offset=0x7e0000
    8192 bytes in size
CPU1 data recorded at offset=0x7e2000
    8192 bytes in size
CPU2 data recorded at offset=0x7e4000
    4096 bytes in size
CPU3 data recorded at offset=0x7e5000
    4096 bytes in size
```

Figure 6.1: Output of the command *trace-cmd record*

```
lorenzo@localhost:/HDD/tesi/dat> sudo trace-cmd report | head -n 10
cpus=4
            CPU-10145 [003]  3729.120309: kvm_exit:             reason MSR_WRITE rip 0xffffffff9727b924 info 0 0
            CPU-10145 [003]  3729.120327: kvm_exit:             reason MSR_WRITE rip 0xffffffff9727b924 info 0 0
            CPU-10145 [003]  3729.120370: kvm_exit:             reason HLT rip 0xffffffff97c0ab6d info 0 0
            CPU-10146 [000]  3729.120450: kvm_exit:             reason MSR_WRITE rip 0xffffffff9727b924 info 0 0
            CPU-10146 [000]  3729.120465: kvm_exit:             reason MSR_WRITE rip 0xffffffff9727b924 info 0 0
            CPU-10146 [000]  3729.120494: kvm_exit:             reason MSR_WRITE rip 0xffffffff9727b924 info 0 0
            CPU-10146 [000]  3729.120501: kvm_exit:             reason HLT rip 0xffffffff97c0ab6d info 0 0
            CPU-10145 [003]  3729.120559: kvm_exit:             reason HLT rip 0xffffffff97c0ab6d info 0 0
            CPU-10145 [003]  3729.129280: kvm_exit:             reason MSR_WRITE rip 0xffffffff9727b924 info 0 0
```

Figure 6.2: Output of the command *trace-cmd report*

The `trace-cmd` command interacts with the *ftrace* tracer that is built inside the Linux kernel. It interfaces with the *ftrace* specific files found in the debugfs

file system under the tracing directory.

A command must be specified to tell `trace-cmd` what to do [5]. The two most important commands are `record` 6.1 and `report` 6.2. Let us see an example, this time by tracing the `kvm_exit` event.

```
$ sudo trace-cmd record -e kvm:kvm_exit
$ sudo trace-cmd report
```

The `record` command starts capturing until we stop it with the usual `Ctrl+c` (interrupting signal). A `trace.dat` file with all the traced information will then be created to the current folder (Figure 6.1 shows the output of `trace-cmd record`). After the recording phase, we can read the `trace.dat` file by the `report` command (Figure 6.2 shows the output of `trace-cmd report`).

6.2 KernelShark

Reading a trace file in text format can be tricky, especially when we are interested in several (and short) time intervals that are far from each other. Furthermore, a short tracing session can result in a considerably large trace file. To get around this, in 2010, **Steven Rostedt** developed *KernelShark*. It is an application which allows the user to graphically analyze the output traces of `trace-cmd` command [3]. Summarizing to interact with ftrace is possible:

Use filesystem It is expensive from the point of view of commands to give, but it is still a primitive way of obtaining information from tracing.

Use trace-cmd It is less expensive to interact directly with the filesystem, but the output is given on the command line, which is not always convenient for analyzing large amounts of data

Use kernelshark Finally, this graphic tool not only allows you to more comfortably analyze the data starting from a `.dat` file generated with `trace-cmd` command, but you can also directly generate the `.dat` file by setting all the options through the graphic menus 6.3.

6.2.1 Usage of KernelShark

As you can see, you can select the plugin (or rather *current_tracer*; in the Figure 6.3 `nop`) and decide to trace just specific events. Therefore, if we select `apply`, on the right side we can see the command which trace-cmd can perform through the command line; in this case:

```
$ sudo trace-cmd record -ekvm:kvm_entry -ekvm:kvm_exit -o trace.dat sleep 0.1
```

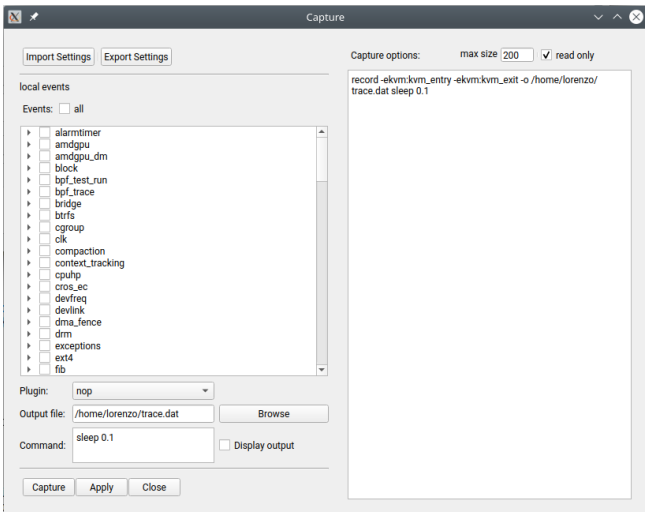


Figure 6.3: Record's tab of KernelShark

Now let us understand some examples of trace analysis. In the first example, we can see a very simple switch (*sched_switch* event); Figures 6.4 6.5 6.6.

In the first step, CPU1, which is running Thread (pid: 7363) decided to waking (`sched_waking`) our InputThread (pid: 1480) on itself (`target_cpu=001`) with low priority (`prio=120`);

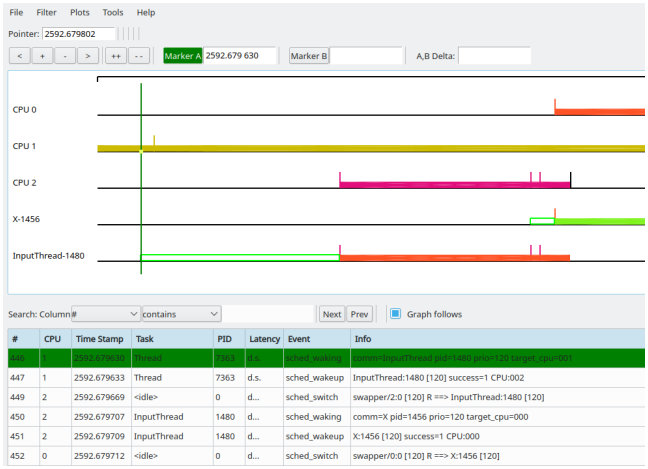


Figure 6.4: Step one of the *sched_switch*

In the second step, the same CPU1 that is running the same process Thread decided to waking up (`sched_wakeup`), this time definitely, the InputThread on CPU2 (`CPU:002`). This choice is probably due to the fact that the CPU1 was

busy running Thread while the second was *idle*;

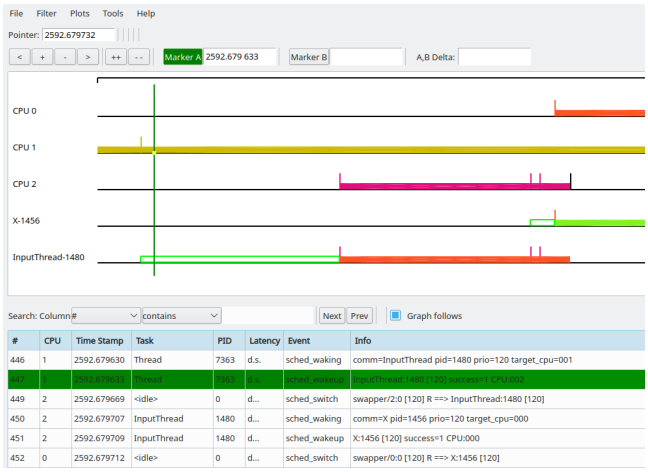


Figure 6.5: Step two of the *sched_switch*

In the last step, finally the CPU2 switch (`sched_switch`) from *idle* to running InputThread.

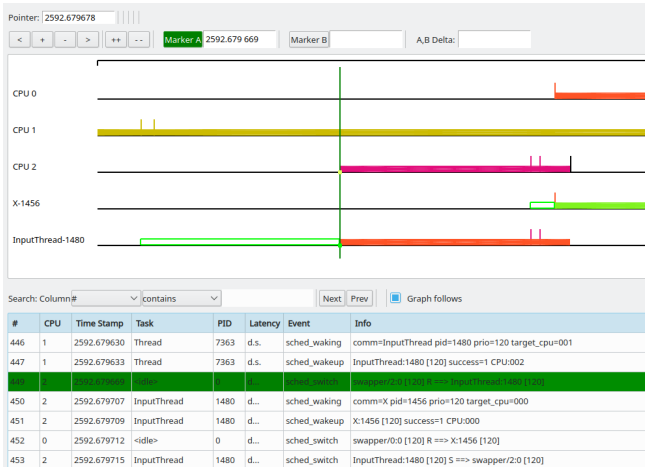


Figure 6.6: Step three of the *sched_switch*

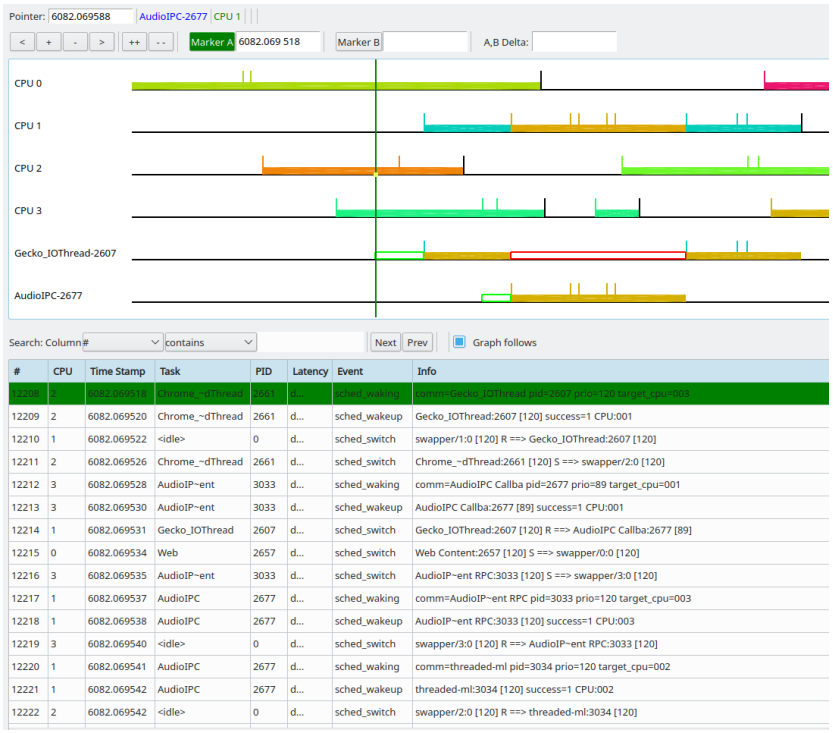
Now that you have become familiar with the GUI, let us present and analyze an example that concerns the events of a preemption; Figure 6.7 In this example, Gecko_IOThread (pid: 2607) would like to run but another process, AudioIPC (pid: 2677) has an higher priority.

1. event number `12210`: Gecko_IOThread start to execute on CPU1

2. event number `12213`: a process (AudioIP) that is running on CPU3 decide to waking up AudioIPC on CPU1

3. event number `12214`: Gecko_IOThread, that would like continue use CPU1, switch in favor of AudioIPC. That happens because AudioIPC has an higher priority (89) compared to Gecko_IOThread (120)

KernelShark helps us to understand what is happening also through graphic conventions. The unfilled red rectangles mean just what is described now. Instead, the unfilled green ones indicate the time from when the process wakes up to when it actually starts to run.



#	CPU	Time Stamp	Task	PID	Latency	Event	Info
12208	2	6082.069518	Chrome_~dThread	2661	d...	sched_waking	comm=Gecko_IOThread pid=2607 prio=120 target_cpu=003
12209	2	6082.069520	Chrome_~dThread	2661	d...	sched_wakeup	Gecko_IOThread:2607 [120] success=1 CPU:001
12210	1	6082.069522	<idle>	0	d...	sched_switch	swapper/1:0 [120] R ==> Gecko_IOThread:2607 [120]
12211	2	6082.069526	Chrome_~dThread	2661	d...	sched_switch	Chrome_~dThread:2661 [120] S ==> swapper/2:0 [120]
12212	3	6082.069528	AudioIP~ent	3033	d...	sched_waking	comm=AudioIPC Callba pid=2677 prio=89 target_cpu=001
12213	3	6082.069530	AudioIP~ent	3033	d...	sched_wakeup	AudioIPC Callba:2677 [89] success=1 CPU:001
12214	1	6082.069531	Gecko_IOThread	2607	d...	sched_switch	Gecko_IOThread:2607 [120] R ==> AudioIPC Callba:2677 [89]
12215	0	6082.069534	Web	2657	d...	sched_switch	Web Content:2657 [120] S ==> swapper/0:0 [120]
12216	3	6082.069535	AudioIP~ent	3033	d...	sched_switch	AudioIP~ent RPC:3033 [120] S ==> swapper/3:0 [120]
12217	1	6082.069537	AudioIPC	2677	d...	sched_waking	comm=AudioIP~ent RPC pid=3033 prio=120 target_cpu=003
12218	1	6082.069538	AudioIPC	2677	d...	sched_wakeup	AudioIP~ent RPC:3033 [120] success=1 CPU:003
12219	3	6082.069540	<idle>	0	d...	sched_switch	swapper/3:0 [120] R ==> AudioIP~ent RPC:3033 [120]
12220	1	6082.069541	AudioIPC	2677	d...	sched_waking	comm=threaded-ml pid=3034 prio=120 target_cpu=002
12221	1	6082.069542	AudioIPC	2677	d...	sched_wakeup	threaded-ml:3034 [120] success=1 CPU:002
12222	2	6082.069542	<idle>	0	d...	sched_switch	swapper/2:0 [120] R ==> threaded-ml:3034 [120]

Figure 6.7: An example of preemption

Finally, let us analyze last example of tracing on the host. It may happen that a process is woken up on one cpu and (during its execution or even before) it is subsequently migrated to another core; that is the case of a `sched_migrate_task` event. In the Figure 6.8 and 6.9, there is an example of this phenomenon.

1. event number `1339`: Chrome is running on CPU3 and decide to waking Gecko_IOThread on CPU1 (6.8)

2. event number `1340`: Chrome also perform the wakeup and confirm the CPU1 for the Gecko_IOThread (6.8)

3. event number `1436`: another process (gfx) that is running in another CPU (2) perform the sched_migrate_task; we can notice that the origin CPU is still CPU1, but the destination CPU is now CPU2 (6.9)

4. shortly after there will be the switch and finally Gecko_IOThread can start its execution on the CPU2

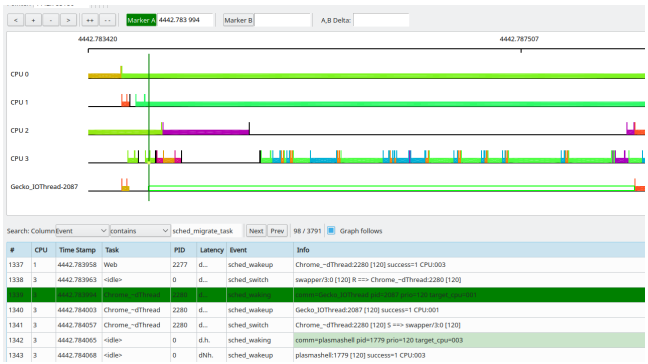


Figure 6.8: Step one migrate task

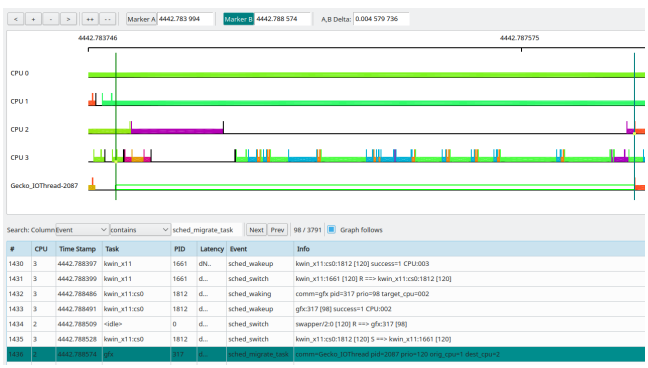


Figure 6.9: Step two, migrate task

We can, also, note that KernelShark allows us to use two markers (A and B). The latter can be useful to understand the time that elapses from one specific event to another without doing anything (see in the figures the label: A, B and *Delta*). If we had to use more primitive interfaces, this work would have cost some time and effort.

Bibliography

[1] Documentation of the linux kernel: Nice levels implementation. https://www.kernel.org/doc/Documentation/scheduler/sched-nice-design.txt.

[2] Extensible scheduler class: Switching to and from sched_ext. https://github.com/sched-ext/sched_ext/blob/sched_ext/Documentation/scheduler/sched-ext.rst#switching-to-and-from-sched_ext.

[3] Kernelshark. https://kernelshark.org/Documentation.html. kernelShark - a front end reader of trace-cmd output.

[4] Overview and motivation of sched_ext. https://github.com/sched-ext/scx/blob/main/OVERVIEW.md#motivation.

[5] trace-cmd. https://linux.die.net/man/1/trace-cmd. trace-cmd - interacts with Ftrace Linux kernel internal tracer.

[6] Gene M Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the April 18-20, 1967, spring joint computer conference*, pages 483–485. ACM, 1967.

[7] Lorenzo Brescia. Coupling tracing over host and guest machines. Master's thesis, University of Turin, December 2020.

[8] Antonio Casoli. Linux scheduler internals: impact of load balancing flags. Master's thesis, University of Turin, July 2020.

[9] Marco Cesati and Daniel P. Bovet. *Understanding the Linux Kernel, Third Edition*. O'reilly, 2005.

[10] Stefano Chiavazza. Linux scheduler internals: Resource management via cgroups. Master's thesis, University of Turin, July 2019.

[11] Francesco Ciraolo. A tool for testing scheduler domains' flags. Master's thesis, University of Turin, July 2020.

[12] Jonathan Corbet. The extensible scheduler class. https://lwn.net/Articles/922405/.

[13] Tejun Heo David Vernet. Scx repository: general overview. `https://github.com/sched-ext/scx/blob/main/OVERVIEW.md`.

[14] Stefano De Venuto. Synchronization accuracy of hypervisor and virtual machine traces. Master's thesis, University of Turin, November 2021.

[15] Mathieu Desnoyers. *Low-Impact Operating System Tracing*. PhD thesis, École Polytechnique de Montréal, 2009.

[16] Giuseppe Eletto. Development of a kernelshark plugin for xen traces analysis. Master's thesis, University of Turin, June 2021.

[17] Matt Fleming. A thorough introduction to ebpf. `https://lwn.net/Articles/740157/`.

[18] Jyothish Jose, Oravanpadath Sujisha, Malayamparambath Gilesh, and Thayyil Bindima. On the fairness of linux o(1) scheduler. In *5th International Conference on Intelligent Systems, Modelling and Simulation*, pages 668–674. IEEE, 2014.

[19] Michael Larabel. Linus torvalds throws down the hammer: Extensible scheduler "sched_ext" in linux 6.11. `https://www.phoronix.com/news/Linux-6.11-Extensible-Scheduler`.

[20] Ingo Molnár. Modular scheduler core and completely fair scheduler. Linux kernel mailing list, `https://www.lwn.net/Articles/230501/`, 2007.

[21] Marco Perronet. Linux kernel: monitoring the scheduler by trace sched* events. Master's thesis, University of Turin, July 2019.

[22] Constantine Shulyupin. Interactive map of linux kernel. `https://makelinux.github.io/kernel/map/`.

[23] Diomidis Spinellis. Trace: A tool for logging operating system call transactions. `https://www2.dmst.aueb.gr/dds/pubs/jrnl/1994-SIGOS-Trace/html/article.html`.

[24] Ion Stoica, Hussein Abdel-Wahab, Kevin Jeffay, Sanjoy K Baruah, Johannes E Gehrke, and C Greg Plaxton. A proportional share resource allocation algorithm for real-time, time-shared systems. In *17th IEEE Real-Time Systems Symposium*, pages 288–299. IEEE, 1996.

[25] Ken Thompson and Dennis Ritchie. Ken Thompson and Dennis Ritchie Explain UNIX (Bell Labs), about 1980. AT&T Bell Labs promotional film.

[26] David Vernet. Kernel recipes 2023 - sched_ext: pluggable scheduling in the linux kernel. `https://www.youtube.com/watch?app=desktop&v=8kAcnNVSAdI`.

[27] David Vernet. Tech over tea: #210 linux kernel scheduler developer. `https://www.youtube.com/watch?v=Ta0imAIz31M`.